

Universidad Nacional Autónoma de México
Instituto de Investigaciones en Matemáticas
Aplicadas y en Sistemas

Licenciatura en Ciencia de Datos

Reconocimiento de Patrones

Práctica 4:
Clasificadores

Alumno:
Carlos Emiliano Mendoza Hernández

Semestre: 2025-2
Fecha: 30/Mayo/2025

Profesores:
Ma. Elena Martínez Pérez
Miguel Angel Veloz Lucas

PRÁCTICA 4: Clasificadores

Tabla de Contenidos

Objetivos	1
Introducción	1
Tipos de Clasificadores para Reconocimiento de Patrones	1
Desarrollo	2
Ejercicio 1: Comparación de clasificadores KNN y DMIN utilizando el conjunto de datos de cáncer de mama.	2
Ejercicio 2: Naive Bayes sobre el conjunto de datos <i>Wine</i>	3
Ejercicio 3: Árboles de decisión sobre el conjunto de subespecies de flor iris	3
Código	5
Ejercicio 1.1: KNN y DMIN con el conjunto de datos de cáncer de mama	5
Ejercicio 1.2: SVM y DMIN con el conjunto de datos de cáncer de mama	6
Ejercicio 2: Naive Bayes sobre el conjunto de datos <i>Wine</i>	7
Ejercicio 3: Árboles de decisión sobre el conjunto de subespecies de flor iris	30
Conclusiones	31
Referencias	31

Objetivos

En esta práctica avanzaremos en el uso de cuatro clasificadores supervisados (KNN, Naive Bayes, Árboles de Decisión y SVM) sobre distintos conjuntos de datos Iris y Wine. Cada ejercicio incluye:

- Preprocesamiento de datos
- Reducción de dimensionalidad PCA y visualización en 2D/3D
- Búsqueda e interpretación de hiperparámetros
- Análisis de resultados

Introducción

Tipos de Clasificadores para Reconocimiento de Patrones

El reconocimiento de patrones es una disciplina fundamental en inteligencia artificial y aprendizaje automático, cuyo objetivo es identificar regularidades o estructuras en datos para asignarles categorías. Los clasificadores son algoritmos que implementan esta tarea, y su elección depende de factores como la naturaleza de los datos, la complejidad del problema y la necesidad de interpretabilidad. A continuación, se describen los principales tipos de clasificadores y sus características.

Clasificadores No Lineales

Cuando los datos presentan relaciones complejas, se emplean modelos no lineales. Los árboles de decisión son un ejemplo: dividen recursivamente el espacio de características mediante reglas jerárquicas, lo que los hace interpretables pero propensos al sobreajuste. Otro enfoque es el algoritmo k-vecinos más cercanos (kNN), que clasifica instancias según la mayoría de clases entre sus vecinos más cercanos. Aunque es flexible y no requiere entrenamiento explícito, su costo computacional crece con el tamaño de los datos. Estos métodos son útiles en reconocimiento de escritura manual o sistemas de recomendación basados en similitud.

Clasificadores Probabilísticos

Basados en teoría de probabilidad, estos modelos estiman distribuciones para predecir clases. El clasificador Naive Bayes es el más conocido: asume independencia entre características y aplica el teorema de Bayes para calcular probabilidades. Aunque simplista, es eficaz en datos categóricos o textuales, como en análisis de sentimientos o detección de spam. Las redes Bayesianas, por otro lado, modelan dependencias entre variables, ofreciendo mayor flexibilidad a costa de complejidad.

Máquina de Soporte Vectorial (SVM)

Las SVM buscan hiperplanos óptimos para separar clases, incluso en espacios de alta dimensión. En su versión lineal, maximizan el margen entre clases, mientras que con kernels transforman los datos a espacios donde la separación no lineal es posible. Son eficaces en reconocimiento facial o clasificación de imágenes médicas, pero requieren ajuste de parámetros y normalización de datos.

Redes Neuronales

Estos modelos, inspirados en el cerebro humano, destacan por su capacidad para aprender patrones complejos. Los perceptrones multicapa (MLP) son redes *feed-forward* con capas ocultas no lineales, mientras que las redes convolucionales (CNN) se especializan en datos grid-like (como imágenes). Las redes recurrentes (RNN) manejan secuencias (texto, series temporales). Aunque son extremadamente poderosas (ej. en visión por computadora o traducción automática), requieren grandes volúmenes de datos, recursos computacionales y carecen de interpretabilidad.

Desarrollo

Ejercicio 1: Comparación de clasificadores KNN y DMIN utilizando el conjunto de datos de cáncer de mama.

El conjunto de datos de cáncer de mama es una clasificación binaria clásica, este set de datos busca determinar si una masa mamaria es un tumor benigno o maligno, a partir de las medidas obtenidas de imágenes digitalizadas de la aspiración con una aguja fina. Los valores representan las características de los núcleos celulares presentes en la imagen digital. La muestra contiene 569 ejemplos de resultados de las biopsias. Cada registro contiene 30 variables, las cuales corresponden a tres medidas (media, desviación estándar, peor caso) de diez características diferentes.

1. Carga y limpieza de datos

- (a) Cargar el conjunto de datos *breast cancer wisconsin* dataset disponible para MATLAB y Python.
- (b) Imprimir el nombre de las características y las etiquetas del conjunto de datos.
- (c) Verificar y aplicar limpieza de datos (faltantes, outliers, caracteres inválidos, etc.).
- (d) Contabilizar el número de muestras malignas y benignas.

2. Preprocesamiento

- (a) Estandarización de características (*zscore()* si utilizas MATLAB y *StandardScaler()* para Python).
- (b) Separación en 70% entrenamiento y 30% prueba.

3. Entrenamiento de modelos de KNN y DMIN

- (a) Entrenar modelo de KNN y probarlo con diferentes valores de k (1, 3, 5, 7, 9).
- (b) Entrenar modelo de DMIN basado en distancia mínima a centroides.

4. Reducción de dimensionalidad

- (a) Obtener la reducción de dimensionalidad del número de componentes para 95% de varianza.
- (b) Generar gráfico de varianza acumulada con línea de referencia al 95%.
- (c) Entrenar los modelos de KNN y DMIN con la nueva distribución de datos.

5. Visualización de resultados

- (a) Mostrar las fronteras de decisión superponiendo datos de entrenamiento y prueba de todos los modelos entrenados con/sin reducción de dimensionalidad PCA.
- (b) Mostrar distribución espacial de datos y posición de centroides en 2 y 3 dimensiones.
- (c) Generar gráfico comparativo de rendimiento sobre las predicciones.

6. Análisis de resultados

- (a) Comparación de exactitud entre de predicciones con/sin reducción de dimensionalidad PCA.
- (b) Matrices de confusión para evaluar errores por clase.

Ejercicio 2: Naive Bayes sobre el conjunto de datos *Wine*

1. Carga y limpieza de Datos:

- (a) Utilizamos el dataset Wine que contiene 13 características químicas de vinos de 3 clases diferentes.
- (b) Verificar y aplicar limpieza de datos (faltantes, outliers, caracteres inválidos, etc.).
- (c) Dividir el conjunto de datos en 75% para entrenamiento y 25% de prueba.

2. Preprocesamiento

- (a) Se implementaran dos tipos de modelos Naive Bayes por lo que el escalado de datos será diferente para cada uno.
- (b) El modelo Bayesiano ingenuo gaussiano asume que los valores continuos asociados a cada característica se distribuyen según una distribución gaussiana.
- (c) El modelo Bayes ingenuo multinomial asume que las características representan la frecuencia de términos, como el número de palabras, requiere datos no negativos.

3. Entrenamiento de modelos de Bayesiano Ingenuo Gaussiano y Bayesiano Ingenuo Multinomial

- (a) Comentar y explicar la selección de hiperparámetros.

4. Visualización de resultados

- (a) Mostrar las fronteras de decisión superponiendo datos de entrenamiento y prueba de todos los modelos entrenados.
- (b) Mostrar distribución espacial de datos en 3 dimensiones.
- (c) Generar gráfico comparativo de rendimiento sobre las predicciones.

5. Análisis de resultados

- (a) Comparación de exactitud entre de predicciones de entrenamiento y prueba.
- (b) Matrices de confusión para evaluar errores por clase.

Ejercicio 3: Árboles de decisión sobre el conjunto de subespecies de flor iris

Esta es quizás la base de datos más conocida que se encuentra en el literatura de reconocimiento de patrones. El conjunto de datos contiene 3 clases de 50 instancias cada una, donde cada clase hace referencia a una subespecie de flor iris.

1. Carga y limpieza de datos

- (a) Utilizamos el conjunto de la flor iris.
- (b) Verificar que no haya valores faltantes y eliminar duplicados

2. Preprocesamiento

- (a) Estandarizar las características escalando los datos.
- (b) Dividir el conjuntos en 80% para entrenamiento y 20% de prueba.

3. Entrenamiento del modelo de Árboles de decisión

- (a) Árbol de decisión con profundidad máxima 3.
- (b) Evaluación con exactitud y matriz de confusión.

4. Visualización de resultados:

- (a) Muestra la estructura del árbol con las reglas de decisión.
- (b) Mostrar las fronteras de decisión superponiendo datos de entrenamiento y prueba.
- (c) Mostrar distribución espacial de datos en 3 dimensiones.

Código

Ejercicio 1.1: KNN y DMIN con el conjunto de datos de cáncer de mama

Practica4_pt1

May 30, 2025

```
[1]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay
from sklearn.impute import SimpleImputer
```

CARGA Y LIMPIEZA DE DATOS

```
[2]: cancer = load_breast_cancer()
X, y = cancer.data, cancer.target

print("Forma del dataset:", X.shape)
print("Cantidad de muestras:", X.shape[0])
print("Cantidad de características:", X.shape[1], end="\n\n")

print("Características:", cancer.feature_names, end="\n\n")
print("Etiquetas:", cancer.target_names, end="\n\n")

# Manejo de valores faltantes
imputer = SimpleImputer(strategy='mean')
X = imputer.fit_transform(X)

unique, counts = np.unique(y, return_counts=True)
for label, count in zip(cancer.target_names, counts):
    print(f"{label}: {count}")
```

Forma del dataset: (569, 30)

Cantidad de muestras: 569

Cantidad de características: 30

Características: ['mean radius' 'mean texture' 'mean perimeter' 'mean area'
'mean smoothness' 'mean compactness' 'mean concavity']

```
'mean concave points' 'mean symmetry' 'mean fractal dimension'
'radius error' 'texture error' 'perimeter error' 'area error'
'smoothness error' 'compactness error' 'concavity error'
'concave points error' 'symmetry error' 'fractal dimension error'
'worst radius' 'worst texture' 'worst perimeter' 'worst area'
'worst smoothness' 'worst compactness' 'worst concavity'
'worst concave points' 'worst symmetry' 'worst fractal dimension']
```

Etiquetas: ['malignant' 'benign']

malignant: 212

benign: 357

PREPROCESAMIENTO DE DATOS

```
[3]: # Separar en conjunto de entrenamiento y prueba (70% train, 30% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42, stratify=y)

# Escalar características
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Tamaño de X_train:", X_train.shape)
print("Tamaño de X_test:", X_test.shape)
print("Tamaño de y_train:", y_train.shape)
print("Tamaño de y_test:", y_test.shape)
```

Tamaño de X_train: (398, 30)

Tamaño de X_test: (171, 30)

Tamaño de y_train: (398,)

Tamaño de y_test: (171,)

IMPLEMENTACION DE DMIN

```
[4]: class DMINClassifier:
    def __init__(self):
        self.centroids = []
        self.classes = []

    def fit(self, X, y):
        self.classes = np.unique(y)
```



```

        self.centroids = [np.mean(X[y == cls], axis=0) for cls in self.classes]

    def predict(self, X):
        return [self.classes[np.argmin([np.linalg.norm(x - centroid) for
↪centroid in self.centroids])] for x in X]

```

ENTRENAMIENTO DE MODELOS

```

[5]: # Entrenar modelos KNN
k_values = [1, 3, 5, 7, 9]
knn_accuracies = []
knn_models = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train) # Usar todas las dimensiones
    y_pred = knn.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    knn_accuracies.append(acc)
    knn_models.append(knn)
    print(f"Precisión KNN (k={k}):", acc)

# Entrenar DMIN
dmin = DMINClassifier()
dmin.fit(X_train, y_train)
y_pred_dmin = dmin.predict(X_test)
acc_dmin = accuracy_score(y_test, y_pred_dmin)
print("Precisión DMIN:", acc_dmin)

```

```

Precisión KNN (k=1): 0.9239766081871345
Precisión KNN (k=3): 0.9181286549707602
Precisión KNN (k=5): 0.9239766081871345
Precisión KNN (k=7): 0.9298245614035088
Precisión KNN (k=9): 0.9415204678362573
Precisión DMIN: 0.8771929824561403

```

```

[6]: # Entrenar modelos KNN
k_values = [1, 3, 5, 7, 9]
knn_accuracies = []
knn_models = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train[:, :2], y_train) # Usar solo 2D para entrenamiento
    y_pred = knn.predict(X_test[:, :2])

```

```

acc = accuracy_score(y_test, y_pred)
knn_accuracies.append(acc)
knn_models.append(knn)
print(f"Precisión KNN (k={k}):", acc)

# Entrenar DMIN
dmin = DMINClassifier()
dmin.fit(X_train[:, :2], y_train)
y_pred_dmin = dmin.predict(X_test[:, :2])
acc_dmin = accuracy_score(y_test, y_pred_dmin)
print("Precisión DMIN:", acc_dmin)

```

```

Precisión KNN (k=1): 0.8245614035087719
Precisión KNN (k=3): 0.8654970760233918
Precisión KNN (k=5): 0.8596491228070176
Precisión KNN (k=7): 0.8888888888888888
Precisión KNN (k=9): 0.8888888888888888
Precisión DMIN: 0.8771929824561403

```

REDUCCIÓN DE DIMENSIONALIDAD

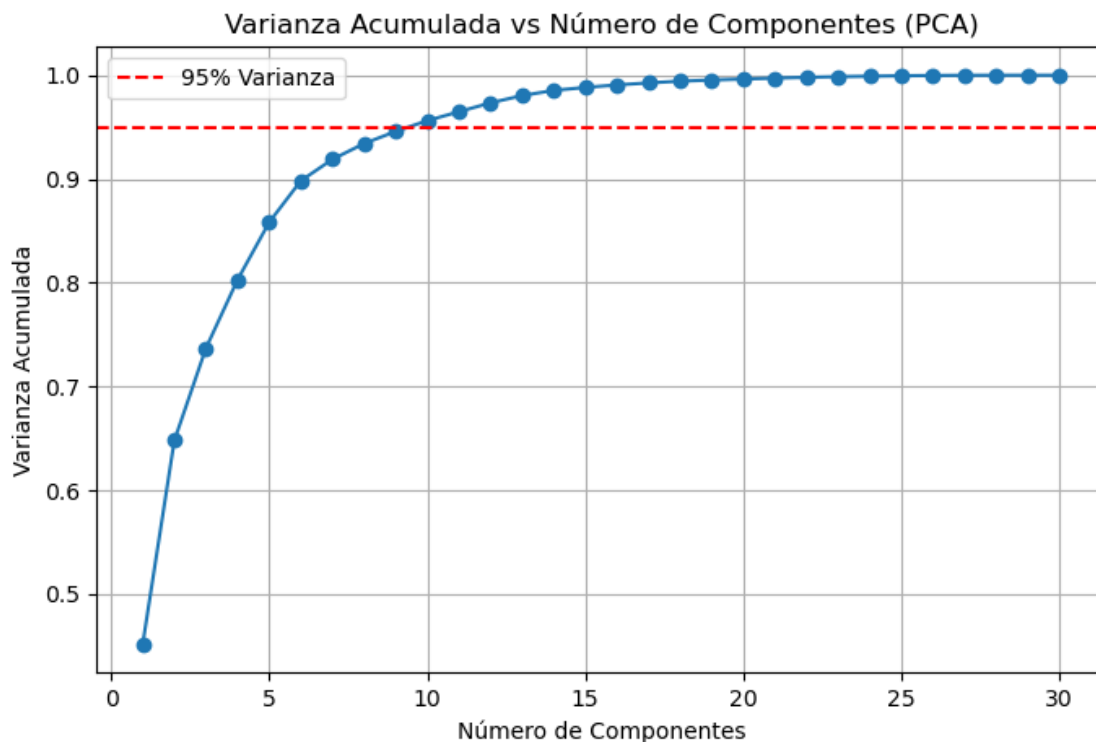
```

[7]: # Aplicar PCA para mantener el 95% de la varianza
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# Graficar la varianza acumulada
plt.figure(figsize=(8, 5))
cum_var = np.cumsum(PCA().fit(X_train_scaled).explained_variance_ratio_)
plt.plot(np.arange(1, len(cum_var) + 1), cum_var, marker='o')
plt.axhline(y=0.95, color='r', linestyle='--', label='95% Varianza')
plt.xlabel('Número de Componentes')
plt.ylabel('Varianza Acumulada')
plt.title('Varianza Acumulada vs Número de Componentes (PCA)')
plt.legend()
plt.grid(True)
plt.show()

print("Número de componentes originales:", X_train_scaled.shape[1])
print("Número de componentes después de PCA:", X_train_pca.shape[1])

```



Número de componentes originales: 30

Número de componentes después de PCA: 10

```
[8]: # Entrenar KNN y DMIN con PCA
knn_pca_accuracies = []
knn_pca_models = []
for k in k_values:
    knn_pca = KNeighborsClassifier(n_neighbors=k)
    knn_pca.fit(X_train_pca, y_train)
    y_pred_pca = knn_pca.predict(X_test_pca)
    acc_pca = accuracy_score(y_test, y_pred_pca)
    knn_pca_accuracies.append(acc_pca)
    knn_pca_models.append(knn_pca)
    print(f"Precisión KNN con PCA (k={k}):", acc_pca)

# Entrenar DMIN con PCA
dmin_pca = DMINClassifier()
dmin_pca.fit(X_train_pca, y_train)
y_pred_dmin_pca = dmin_pca.predict(X_test_pca)
acc_dmin_pca = accuracy_score(y_test, y_pred_dmin_pca)
print("Precisión DMIN con PCA:", acc_dmin_pca)
```

Precisión KNN con PCA (k=1): 0.9590643274853801

Precisión KNN con PCA (k=3): 0.9415204678362573

Precisión KNN con PCA (k=5): 0.9532163742690059
Precisión KNN con PCA (k=7): 0.9590643274853801
Precisión KNN con PCA (k=9): 0.9590643274853801
Precisión DMIN con PCA: 0.9298245614035088

```
[9]: # PCA con visualización en 2D para visualización
pca_vis = PCA(n_components=2)
X_train_pca_vis = pca_vis.fit_transform(X_train_scaled)
X_test_pca_vis = pca_vis.transform(X_test_scaled)

# Entrenar KNN y DMIN con PCA
knn_pca_accuracies = []
knn_pca_models = []
for k in k_values:
    knn_pca = KNeighborsClassifier(n_neighbors=k)
    knn_pca.fit(X_train_pca_vis, y_train)
    y_pred_pca = knn_pca.predict(X_test_pca_vis)
    acc_pca = accuracy_score(y_test, y_pred_pca)
    knn_pca_accuracies.append(acc_pca)
    knn_pca_models.append(knn_pca)
    print(f"Precisión KNN con PCA (k={k}):", acc_pca)

# Entrenar DMIN con PCA
dmin_pca = DMINClassifier()
dmin_pca.fit(X_train_pca_vis, y_train)
y_pred_dmin_pca = dmin_pca.predict(X_test_pca_vis)
acc_dmin_pca = accuracy_score(y_test, y_pred_dmin_pca)
print("Precisión DMIN con PCA:", acc_dmin_pca)
```

Precisión KNN con PCA (k=1): 0.9122807017543859
Precisión KNN con PCA (k=3): 0.9415204678362573
Precisión KNN con PCA (k=5): 0.9415204678362573
Precisión KNN con PCA (k=7): 0.9473684210526315
Precisión KNN con PCA (k=9): 0.9473684210526315
Precisión DMIN con PCA: 0.935672514619883

=====

VISUALIZACIÓN DE RESULTADOS

=====

```
[10]: def plot_decision_boundaries(model, X, y, title, ax, X_test=None, y_test=None):
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                          np.linspace(y_min, y_max, 100))

    if isinstance(model, KNeighborsClassifier):
```

```

        Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    else:
        Z = np.array(model.predict(np.c_[xx.ravel(), yy.ravel()])))

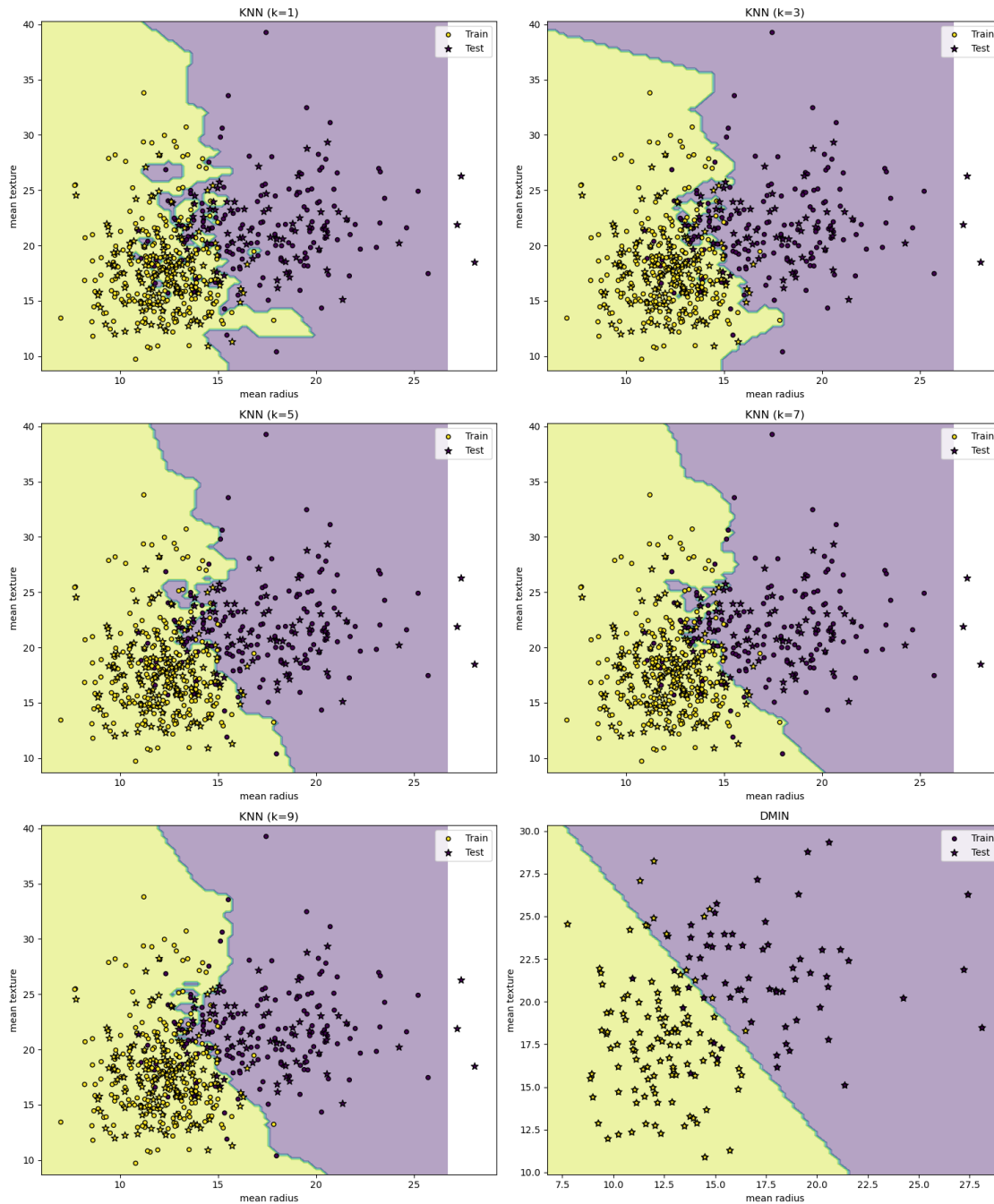
    Z = Z.reshape(xx.shape)
    ax.contourf(xx, yy, Z, alpha=0.4)
    ax.scatter(X[:, 0], X[:, 1], c=y, s=20, edgecolor='k', label='Train')
    if X_test is not None and y_test is not None:
        ax.scatter(X_test[:, 0], X_test[:, 1], c=y_test, s=60, marker='*',
        ↪edgecolor='k', label='Test')
    ax.set_title(title)
    ax.legend()

# Visualización de fronteras de decisión para cada modelo KNN, incluyendo test
fig, axes = plt.subplots(3, 2, figsize=(15, 18))
axes = axes.flatten()
for idx, model in enumerate(knn_models):
    plot_decision_boundaries(
        model,
        X_train[:, :2],
        y_train,
        f"KNN (k={k_values[idx]})",
        axes[idx],
        X_test=X_test[:, :2],
        y_test=y_test
    )
    axes[idx].set_xlabel(cancer.feature_names[0])
    axes[idx].set_ylabel(cancer.feature_names[1])

plot_decision_boundaries(dmin, X_test[:, :2], y_test, 'DMIN', axes[5],
    ↪X_test=X_test[:, :2], y_test=y_test)
axes[5].set_xlabel(cancer.feature_names[0])
axes[5].set_ylabel(cancer.feature_names[1])

plt.tight_layout()
plt.show()

```



```
[11]: # PCA solo para visualización (2 componentes)
pca_vis = PCA(n_components=2)
X_train_pca_vis = pca_vis.fit_transform(X_train_scaled)
X_test_pca_vis = pca_vis.transform(X_test_scaled)

# Entrenar modelos con solo 2 componentes
```

```

knn_pca_vis_models = []
for k in k_values:
    knn_pca_vis = KNeighborsClassifier(n_neighbors=k)
    knn_pca_vis.fit(X_train_pca_vis, y_train)
    knn_pca_vis_models.append(knn_pca_vis)

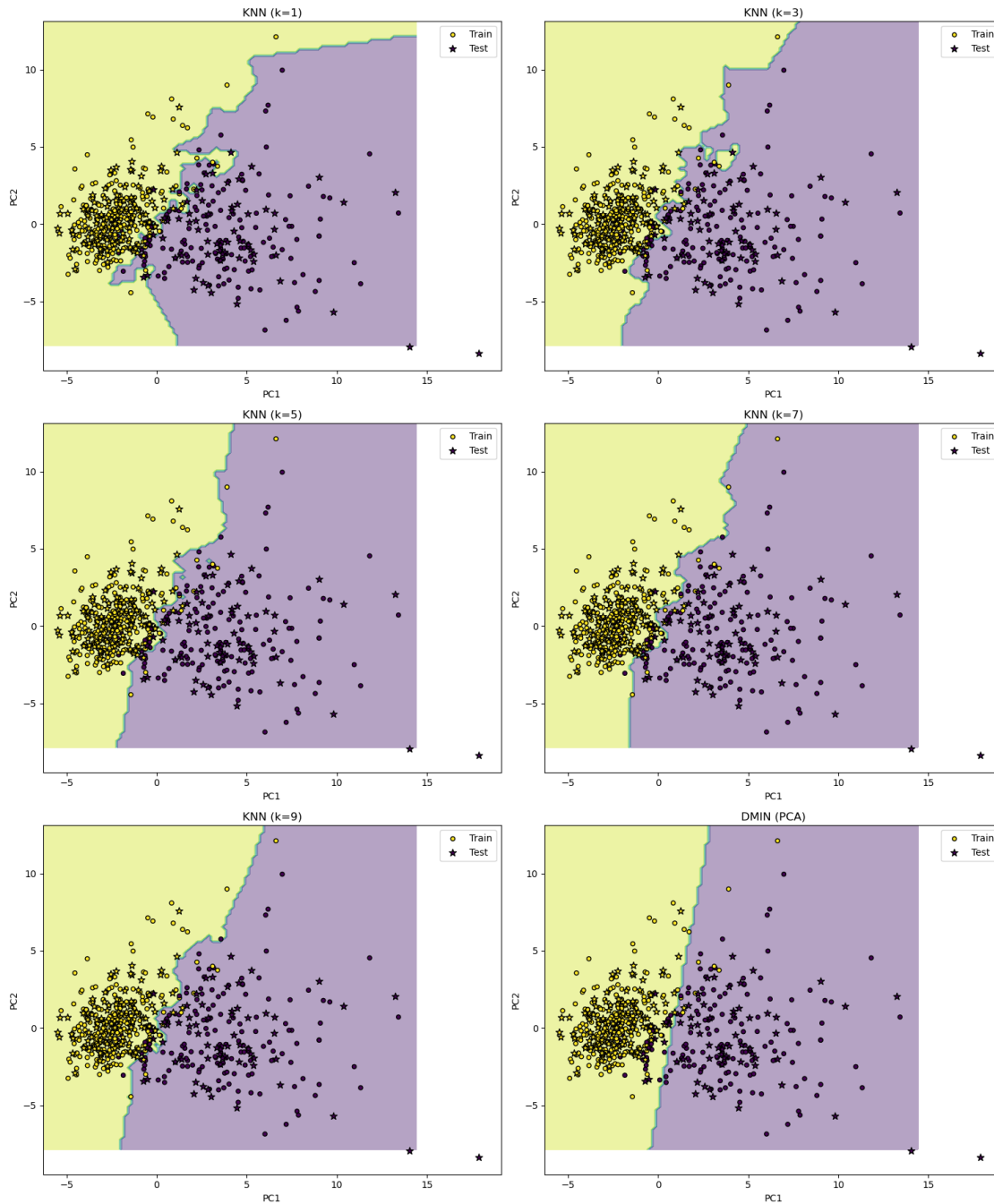
# Entrenar DMIN con PCA para visualización
dmin_pca_vis = DMINClassifier()
dmin_pca_vis.fit(X_train_pca_vis, y_train)
y_pred_dmin_pca_vis = dmin_pca_vis.predict(X_test_pca_vis)

# Visualización
fig, axes = plt.subplots(3, 2, figsize=(15, 18))
axes = axes.flatten()
for idx, model in enumerate(knn_pca_vis_models):
    plot_decision_boundaries(
        model,
        X_train_pca_vis,
        y_train,
        f"KNN (k={k_values[idx]})",
        axes[idx],
        X_test=X_test_pca_vis,
        y_test=y_test
    )
    axes[idx].set_xlabel('PC1')
    axes[idx].set_ylabel('PC2')

plot_decision_boundaries(dmin_pca_vis, X_train_pca_vis, y_train, 'DMIN (PCA)',
    axes[5], X_test=X_test_pca_vis, y_test=y_test)
axes[5].set_xlabel('PC1')
axes[5].set_ylabel('PC2')

plt.tight_layout()
plt.show()

```



```
[12]: # Visualizacion 3D
fig = plt.figure(figsize=(18, 6))

# KNN 3D
ax1 = fig.add_subplot(131, projection='3d')
```



```

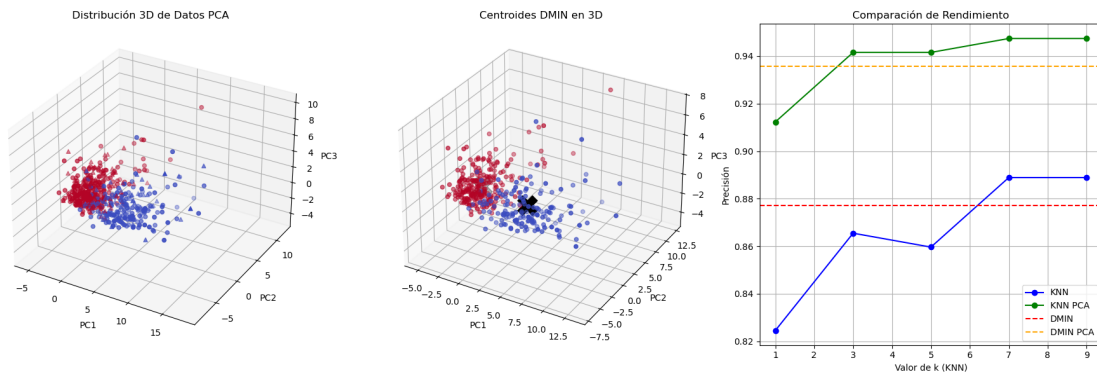
ax1.scatter(X_train_pca[:, 0], X_train_pca[:, 1], X_train_pca[:, 2], c=y_train,
            cmap='coolwarm', label='Entrenamiento')
ax1.scatter(X_test_pca[:, 0], X_test_pca[:, 1], X_test_pca[:, 2], c=y_test,
            cmap='coolwarm', marker='^', label='Prueba')
ax1.set_title('Distribución 3D de Datos PCA')
ax1.set_xlabel('PC1')
ax1.set_ylabel('PC2')
ax1.set_zlabel('PC3')

# DMIN 3D
ax2 = fig.add_subplot(132, projection='3d')
ax2.scatter(X_train_pca[:, 0], X_train_pca[:, 1], X_train_pca[:, 2], c=y_train,
            cmap='coolwarm')
ax2.scatter(np.array(dmin_pca.centroids)[:, 0],
            np.array(dmin_pca.centroids)[:, 1],
            [0, 0], s=500, marker='X', color='black')
ax2.set_title('Centroides DMIN en 3D')
ax2.set_xlabel('PC1')
ax2.set_ylabel('PC2')
ax2.set_zlabel('PC3')

# Comparación de precisión
ax3 = fig.add_subplot(133)
ax3.plot(k_values, knn_accuracies, 'bo-', label='KNN')
ax3.plot(k_values, knn_pca_accuracies, 'go-', label='KNN PCA')
ax3.axhline(acc_dmin, color='r', linestyle='--', label='DMIN')
ax3.axhline(acc_dmin_pca, color='orange', linestyle='--', label='DMIN PCA')
ax3.set_xlabel('Valor de k (KNN)')
ax3.set_ylabel('Precisión')
ax3.set_title('Comparación de Rendimiento')
ax3.legend()
ax3.grid(True)

plt.tight_layout()
plt.show()

```

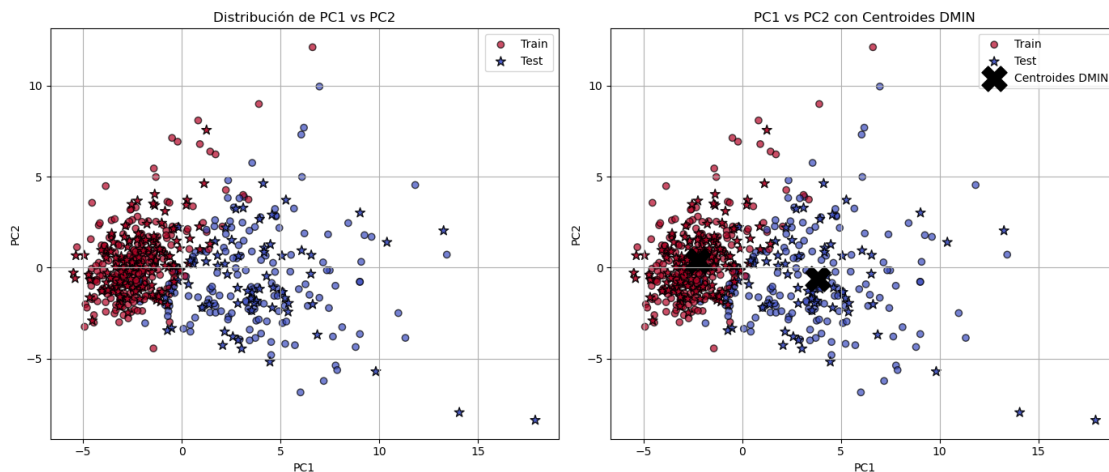


```
[13]: fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Scatter plot of PC1 vs PC2 (train and test set)
axes[0].scatter(X_train_pca_vis[:, 0], X_train_pca_vis[:, 1], c=y_train,
               ↪ cmap='coolwarm', edgecolor='k', alpha=0.7, marker='o', label='Train')
axes[0].scatter(X_test_pca_vis[:, 0], X_test_pca_vis[:, 1], c=y_test,
               ↪ cmap='coolwarm', edgecolor='k', alpha=0.9, marker='*', s=80, label='Test')
axes[0].set_title('Distribución de PC1 vs PC2')
axes[0].set_xlabel('PC1')
axes[0].set_ylabel('PC2')
axes[0].legend()
axes[0].grid(True)

# Scatter plot with DMIN centroids
axes[1].scatter(X_train_pca_vis[:, 0], X_train_pca_vis[:, 1], c=y_train,
               ↪ cmap='coolwarm', edgecolor='k', alpha=0.7, marker='o', label='Train')
axes[1].scatter(X_test_pca_vis[:, 0], X_test_pca_vis[:, 1], c=y_test,
               ↪ cmap='coolwarm', edgecolor='k', alpha=0.9, marker='*', s=80, label='Test')
centroids = np.array(dmin_pca_vis.centroids)
axes[1].scatter(centroids[:, 0], centroids[:, 1], c='black', marker='X', s=500,
               ↪ label='Centroides DMIN')
axes[1].set_title('PC1 vs PC2 con Centroides DMIN')
axes[1].set_xlabel('PC1')
axes[1].set_ylabel('PC2')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```



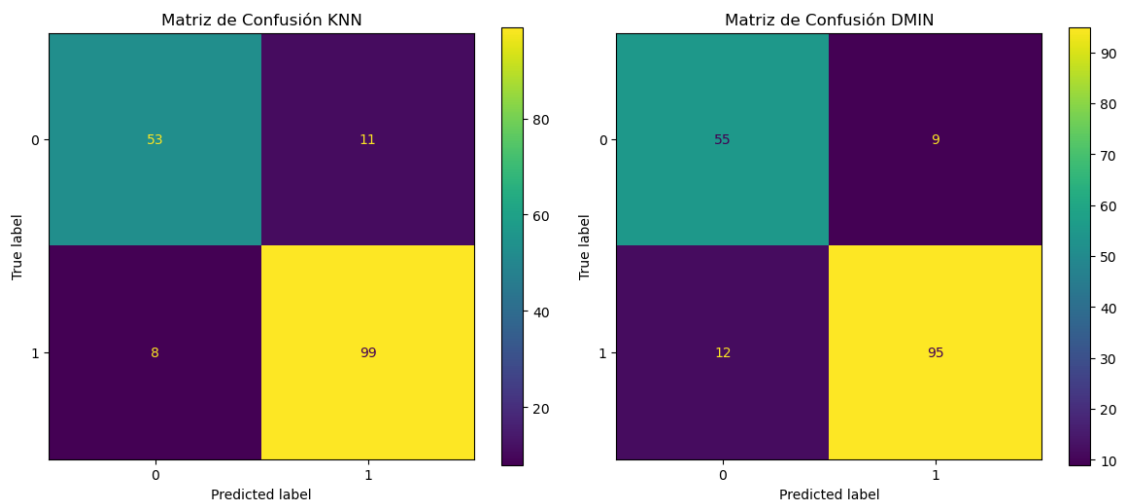
```
[14]: knn_best = knn_models[np.argmax(knn_pca_accuracies)]

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ConfusionMatrixDisplay.from_estimator(knn_best, X_test[:, :2], y_test, ax=ax[0])
ax[0].set_title('Matriz de Confusión KNN')

ConfusionMatrixDisplay.from_predictions(y_test, y_pred_dmin, ax=ax[1])
ax[1].set_title('Matriz de Confusión DMIN')

plt.tight_layout()
plt.show()

print(f"\n{'='*40}\nResultados Finales:\n{'='*40}")
print(f"Mejor KNN con k={k_values[np.argmax(knn_pca_accuracies)]}")
print(f"Exactitud = {max(knn_accuracies):.4f}")
print(f"DMIN: Exactitud = {acc_dmin_pca:.4f}")
print(f"Varianza explicada por componentes PCA: {pca.explained_variance_ratio_[:3]}")
```



```
=====
Resultados Finales:
=====
Mejor KNN con k=7
Exactitud = 0.8889
DMIN: Exactitud = 0.9357
Varianza explicada por componentes PCA: [0.45156229 0.19628669 0.08897898]
```

Ejercicio 1.2: SVM y DMIN con el conjunto de datos de cáncer de mama

Practica4_pt2

May 30, 2025

```
[2]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix
```

ENTRENAMIENTO SVM

```
[3]: # 1. Cargar y preparar los datos
data = load_breast_cancer()
X, y = data.data, data.target

# Estandarizacion
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Reducir a 2 dimensiones usando PCA
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)

# Separar los datos en conjuntos de entrenamiento y prueba (70% entrenamiento, 30% prueba)
X_train, X_test, y_train, y_test = train_test_split(X_pca, y, test_size=0.3, random_state=42)

# 2. Entrenamiento y evaluación de modelos SVM con diferentes hiperparámetros
hyperparams = [
    {'C': 0.1, 'kernel': 'linear'},
    {'C': 5, 'kernel': 'linear'},
    {'C': 10, 'kernel': 'linear'}
```

```

]

models = []
scores = []
conf_matrices = []

for params in hyperparams:
    svm = SVC(**params, random_state=42)
    svm.fit(X_train, y_train)
    y_pred = svm.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)

    models.append((svm, params))
    scores.append(acc)
    conf_matrices.append(cm)

```

=====

VISUALIZACION DE RESULTADOS

=====

```

[4]: # 3a. Fronteras de decisión y puntos de entrenamiento/prueba
fig, axes = plt.subplots(1, len(models), figsize=(18, 5))

# Crear una malla para las fronteras de decisión
x_min, x_max = X_pca[:, 0].min() - 1, X_pca[:, 0].max() + 1
y_min, y_max = X_pca[:, 1].min() - 1, X_pca[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 500),
                     np.linspace(y_min, y_max, 500))
grid_points = np.c_[xx.ravel(), yy.ravel()]

for idx, (svm, params) in enumerate(models):
    ax = axes[idx]
    # Prediccion de la malla
    Z = svm.predict(grid_points).reshape(xx.shape)
    ax.contourf(xx, yy, Z, alpha=0.2)

    # Graficar puntos de entrenamiento y prueba
    ax.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap='coolwarm',
               edgecolors='k', marker='o', label='Train')
    ax.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap='coolwarm',
               edgecolors='k', marker='s', label='Test')

    ax.set_title(f"SVM (C={params['C']}, kernel={params['kernel']})\nAcc: {scores[idx]:.2f}")
    ax.set_xlabel('PCA 1')

```

```

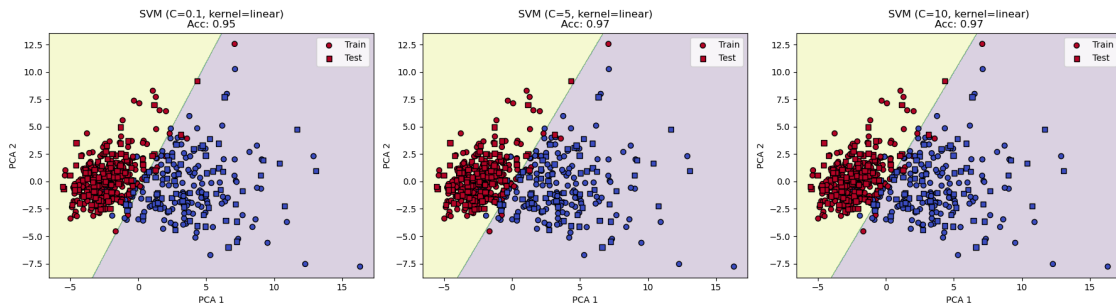
ax.set_ylabel('PCA 2')
ax.legend(loc='upper right')

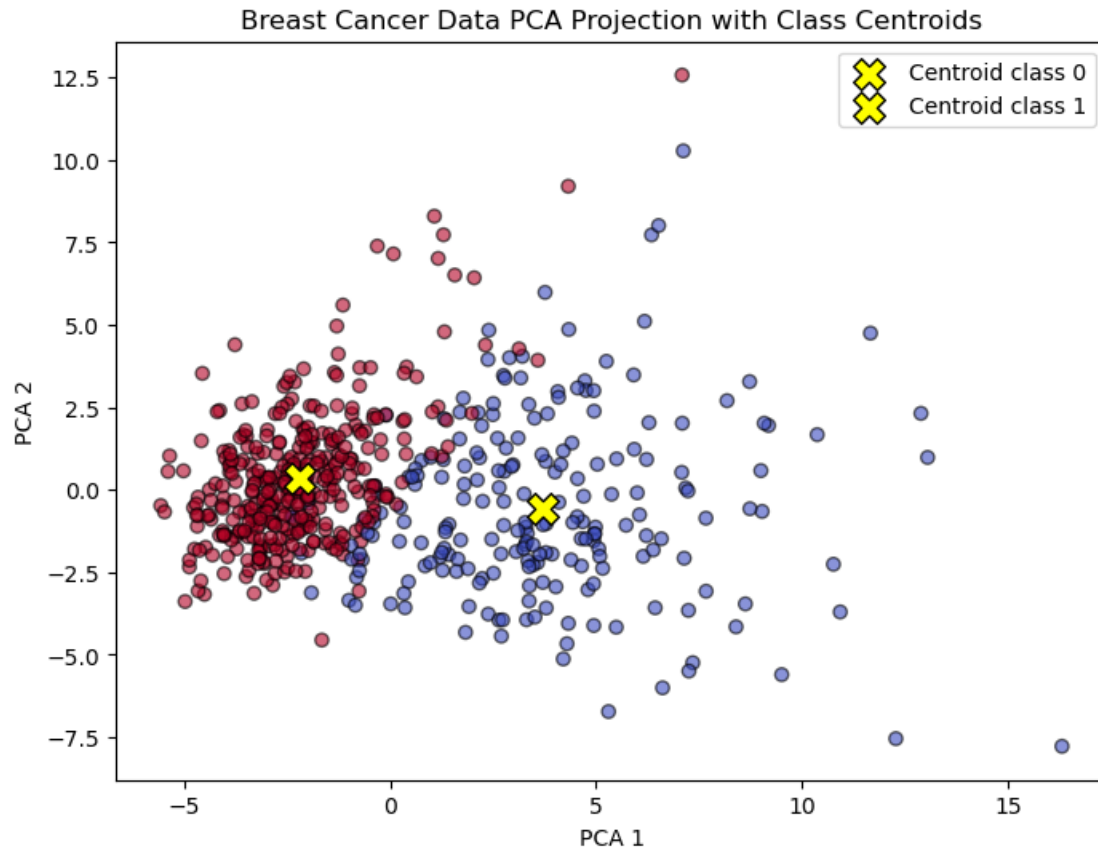
plt.tight_layout()

# 3b. Distribución de clases y centroides
fig, ax = plt.subplots(figsize=(8, 6))
# Graficas los puntos de datos en el espacio PCA
ax.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='coolwarm', edgecolors='k',
           alpha=0.6)
# Calcular y graficar los centroides de cada clase
centroids = []
for label in np.unique(y):
    mean_point = X_pca[y == label].mean(axis=0)
    centroids.append(mean_point)
    ax.scatter(mean_point[0], mean_point[1], c='yellow', edgecolors='k', s=200,
               marker='X', label=f"Centroid class {label}")

ax.set_title('Breast Cancer Data PCA Projection with Class Centroids')
ax.set_xlabel('PCA 1')
ax.set_ylabel('PCA 2')
ax.legend()
plt.show()

```





=====

Analisis de Resultados

=====

```
[5]: # Gráfica de precisión vs. parámetro C
fig, ax = plt.subplots(figsize=(6, 4))
param_labels = [f"C={p['C']}" for p in hyperparams]
ax.bar(param_labels, scores)
ax.set_title('Comparison of SVM Accuracy for Different C Values')
ax.set_xlabel('Hyperparameter Setting')
ax.set_ylabel('Accuracy')
ax.set_ylim(0, 1)

for i, acc in enumerate(scores):
    ax.text(i, acc, f"{acc:.2f}", ha='center', va='bottom', fontsize=10)
plt.show()

# Graficar cada matriz de confusión como heatmap
num_matrices = len(conf_matrices)
```



```

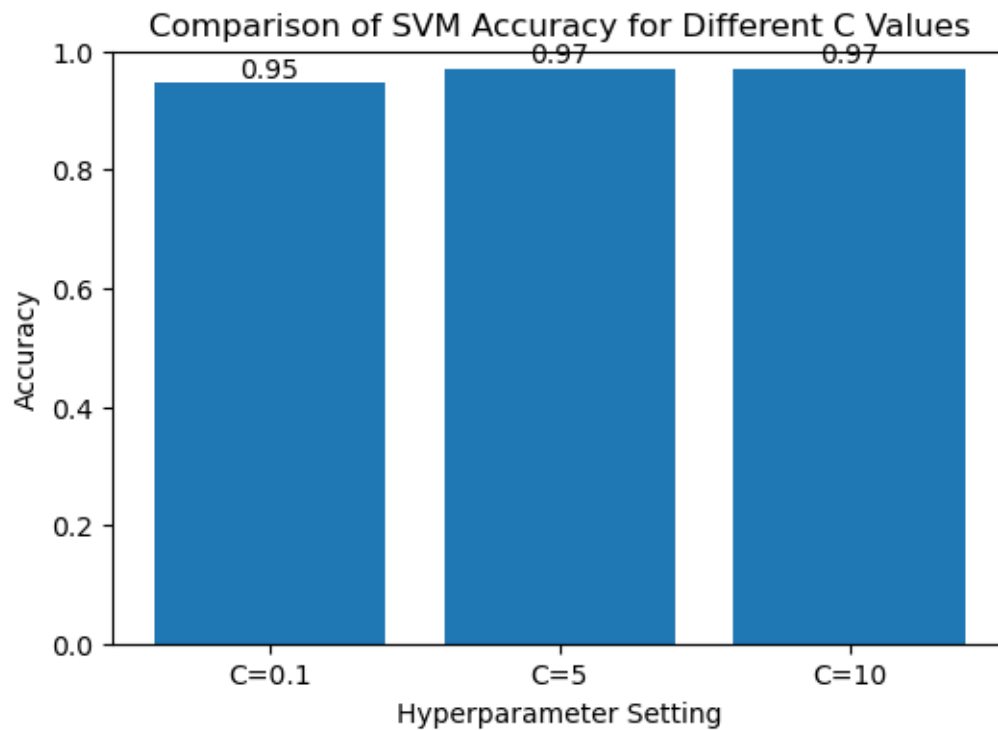
fig, axes = plt.subplots(1, num_matrices, figsize=(5 * num_matrices, 4))

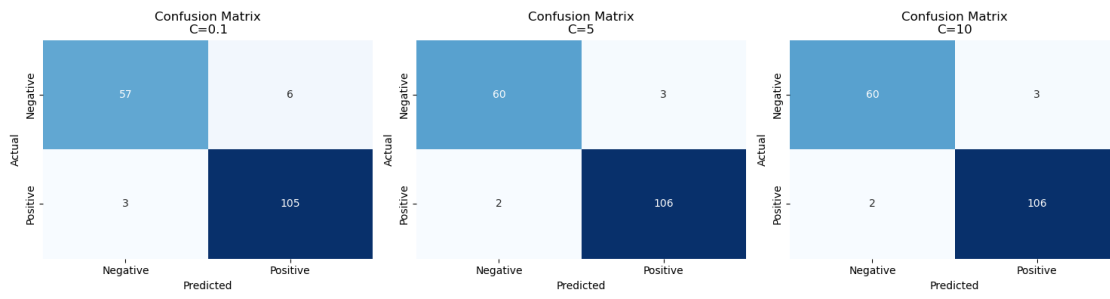
if num_matrices == 1:
    axes = [axes]

for ax, cm, param in zip(axes, conf_matrices, hyperparams):
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax, cbar=False)
    ax.set_title(f"Confusion Matrix\nC={param['C']}")
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")
    ax.set_xticklabels(['Negative', 'Positive'])
    ax.set_yticklabels(['Negative', 'Positive'])

plt.tight_layout()
plt.show()

```





Ejercicio 2: Naive Bayes sobre el conjunto de datos *Wine*

```
clear; close all; clc;  
format shortG;
```

```
%% =====  
%% CARGA Y EXPLORACIÓN INICIAL DE DATOS  
%% =====  
wine = readtable('./data/Wine.csv');  
[n_samples, n_features] = size(wine);  
fprintf('Dataset cargado: %d muestras, %d características\n', n_samples, n_features-1);
```

Dataset cargado: 178 muestras, 13 características

```
% Separar características y etiquetas  
X = wine(:, 1:13);    % Características  
y = wine(:, 14);      % Etiquetas  
  
% Información básica del dataset  
fprintf('Clases únicas: %s\n', mat2str(unique(y)'));
```

Clases únicas: [1 2 3]

```
fprintf('Distribución de clases:\n');
```

Distribución de clases:

```
for class = unique(y)  
    fprintf(' Clase %d: %d muestras (%.1f%%)\n', class, sum(y==class), 100*sum(y==class)/length(y));  
end
```

Clase 1: 59 muestras (33.1%)
Clase 2: 71 muestras (39.9%)
Clase 3: 48 muestras (27.0%)

```
%% =====  
%% LIMPIEZA DE DATOS  
%% =====  
fprintf('\n=== LIMPIEZA DE DATOS ===\n');
```

=== LIMPIEZA DE DATOS ===

```
% 1. Detección y eliminación de valores faltantes  
missingRows = any(ismissing(wine), 2);  
if any(missingRows)  
    fprintf('Eliminando %d filas con datos faltantes...\n', sum(missingRows));  
    X(missingRows, :) = [];  
    y(missingRows) = [];  
else
```

```
    fprintf('No se encontraron valores faltantes.\n');
end
```

No se encontraron valores faltantes.

```
% 2. Detección y eliminación de outliers (método IQR)
fprintf('Detectando outliers usando método IQR...\n');
```

Detectando outliers usando método IQR...

```
original_size = size(X,1);
outlierRows = false(size(X, 1), 1);
for i = 1:size(X, 2)
    Q1 = quantile(X(:, i), 0.25);
    Q3 = quantile(X(:, i), 0.75);
    IQR = Q3 - Q1;
    lower_bound = Q1 - 1.5 * IQR;
    upper_bound = Q3 + 1.5 * IQR;
    outlierRows = outlierRows | (X(:, i) < lower_bound) | (X(:, i) > upper_bound);
end

if any(outlierRows)
    fprintf('Eliminando %d filas con outliers...\n', sum(outlierRows));
    X(outlierRows, :) = [];
    y(outlierRows) = [];
else
    fprintf('No se encontraron outliers significativos.\n');
end
```

Eliminando 16 filas con outliers...

```
% 3. Verificación de tipos de datos
if ~isnumeric(X) || ~isnumeric(y)
    error('Se encontraron valores no numéricos en X o en y después de la limpieza.');
```

```
end

fprintf('Datos después de limpieza: %d muestras (%.1f%% del original)\n', ...
        size(X,1), 100*size(X,1)/original_size);
```

Datos después de limpieza: 162 muestras (91.0% del original)

```
% Escalado Z-score
X_scaled = zscore(X);

% Reducción dimensional (PCA)
[~, score] = pca(X_scaled);
X_pca = score(:,1:3);

% División entrenamiento/prueba
rng(42);
cv = cvpartition(y, 'HoldOut', 0.25);
X_train = X_pca(cv.training,:);
```

```

y_train = y(cv.training);
X_test = X_pca(cv.test,:);
y_test = y(cv.test);

fprintf('Conjunto de entrenamiento: %d muestras\n', size(X_train, 1));

```

Conjunto de entrenamiento: 122 muestras

```

fprintf('Conjunto de prueba: %d muestras\n', size(X_test, 1));

```

Conjunto de prueba: 40 muestras

```

% Verificar distribución de clases en ambos conjuntos
fprintf('\nDistribución en entrenamiento:\n');

```

Distribución en entrenamiento:

```

for class = unique(y_train)'
    fprintf(' Clase %d: %d muestras (%.1f%%)\n', class, sum(y_train==class), 100*sum(y_train==class)/length(y_train));
end

```

```

Clase 1: 43 muestras (35.2%)
Clase 2: 46 muestras (37.7%)
Clase 3: 33 muestras (27.0%)

```

```

fprintf('Distribución en prueba:\n');

```

Distribución en prueba:

```

for class = unique(y_test)'
    fprintf(' Clase %d: %d muestras (%.1f%%)\n', class, sum(y_test==class), 100*sum(y_test==class)/length(y_test));
end

```

```

Clase 1: 15 muestras (37.5%)
Clase 2: 15 muestras (37.5%)
Clase 3: 10 muestras (25.0%)

```

```

%% =====
%% PREPROCESAMIENTO PARA MULTINOMIAL NAIVE BAYES
%% =====

```

```

% El Multinomial NB requiere valores no negativos (frecuencias)
% Método 1: Min-Max con rango global (considerando train y test)
fprintf('Calculando rango global para transformación Min-Max...\n');

```

Calculando rango global para transformación Min-Max...

```

X_combined = [X_train; X_test]; % Combinar para obtener rango global
min_vals_global = min(X_combined, [], 1);
max_vals_global = max(X_combined, [], 1);
range_vals_global = max_vals_global - min_vals_global;

% Evitar división por cero
range_vals_global(range_vals_global == 0) = 1;

% Normalización Min-Max usando rango global
X_train_norm = (X_train - min_vals_global) ./ range_vals_global;
X_test_norm = (X_test - min_vals_global) ./ range_vals_global;

% Verificar rangos después de normalización
fprintf('Rango en train después de Min-Max: [%.6f, %.6f]\n', min(X_train_norm(:)), max(X_train_norm(:)));

```

```

Rango en train después de Min-Max: [0.000000, 1.000000]

```

```

fprintf('Rango en test después de Min-Max: [%.6f, %.6f]\n', min(X_test_norm(:)), max(X_test_norm(:)));

```

```

Rango en test después de Min-Max: [0.000000, 0.963701]

```

```

% Método alternativo si aún hay problemas: Usar transformación de desplazamiento
if min(X_test_norm(:)) < 0 || min(X_train_norm(:)) < 0
    fprintf('Detectados valores negativos, aplicando método de desplazamiento...\n');

    % Encontrar el valor mínimo en todo el dataset
    min_value_all = min([X_train(:); X_test(:)]);

    % Desplazar todos los valores para que el mínimo sea 0
    X_train_shifted = X_train - min_value_all;
    X_test_shifted = X_test - min_value_all;

    % Aplicar Min-Max a los datos desplazados
    max_shifted = max([X_train_shifted(:); X_test_shifted(:)]);
    X_train_norm = X_train_shifted / max_shifted;
    X_test_norm = X_test_shifted / max_shifted;

    fprintf('Método de desplazamiento aplicado.\n');
    fprintf('Nuevo rango en train: [%.6f, %.6f]\n', min(X_train_norm(:)), max(X_train_norm(:)));
    fprintf('Nuevo rango en test: [%.6f, %.6f]\n', min(X_test_norm(:)), max(X_test_norm(:)));
end

% Escalar a rango de frecuencias enteras (multiplicar por 100 y redondear)
scaling_factor = 100;
X_train_multinom = round(X_train_norm * scaling_factor);
X_test_multinom = round(X_test_norm * scaling_factor);

% Asegurar que no hay valores negativos (forzar a 0 si los hay)
X_train_multinom = max(X_train_multinom, 0);
X_test_multinom = max(X_test_multinom, 0);

```

```
% Verificación final
if any(X_train_multinom(:) < 0) || any(X_test_multinom(:) < 0)
    error('Error: Aún hay valores negativos después de todas las transformaciones');
else
    fprintf(' Transformación exitosa: todos los valores son no negativos\n');
end
```

Transformación exitosa: todos los valores son no negativos

```
fprintf('Datos preparados para Multinomial NB:\n');
```

Datos preparados para Multinomial NB:

```
fprintf(' Rango de valores en entrenamiento: [%d, %d]\n', ...
        min(X_train_multinom(:)), max(X_train_multinom(:)));
```

Rango de valores en entrenamiento: [0, 100]

```
%% =====
%% VISUALIZACIÓN DEL PREPROCESAMIENTO
%% =====

% Comparación de distribuciones antes y después del preprocesamiento
figure('Position', [100, 100, 1200, 800]);

% Datos originales
subplot(2, 3, 1);
histogram(X(:), 30, 'Normalization', 'probability');
title('Datos Originales');
xlabel('Valor');
ylabel('Probabilidad');

% Datos estandarizados (Gaussian)
subplot(2, 3, 2);
histogram(X_train(:), 30, 'Normalization', 'probability');
title('Datos Estandarizados (Gaussian NB)');
xlabel('Valor');
ylabel('Probabilidad');

% Datos para Multinomial
subplot(2, 3, 3);
histogram(X_train_multinom(:), 30, 'Normalization', 'probability');
title('Datos Transformados (Multinomial NB)');
xlabel('Valor');
ylabel('Probabilidad');

% Boxplots de comparación
subplot(2, 3, 4);
boxplot(X(:, 1:3));
title('Boxplot Datos Originales');

subplot(2, 3, 5);
```

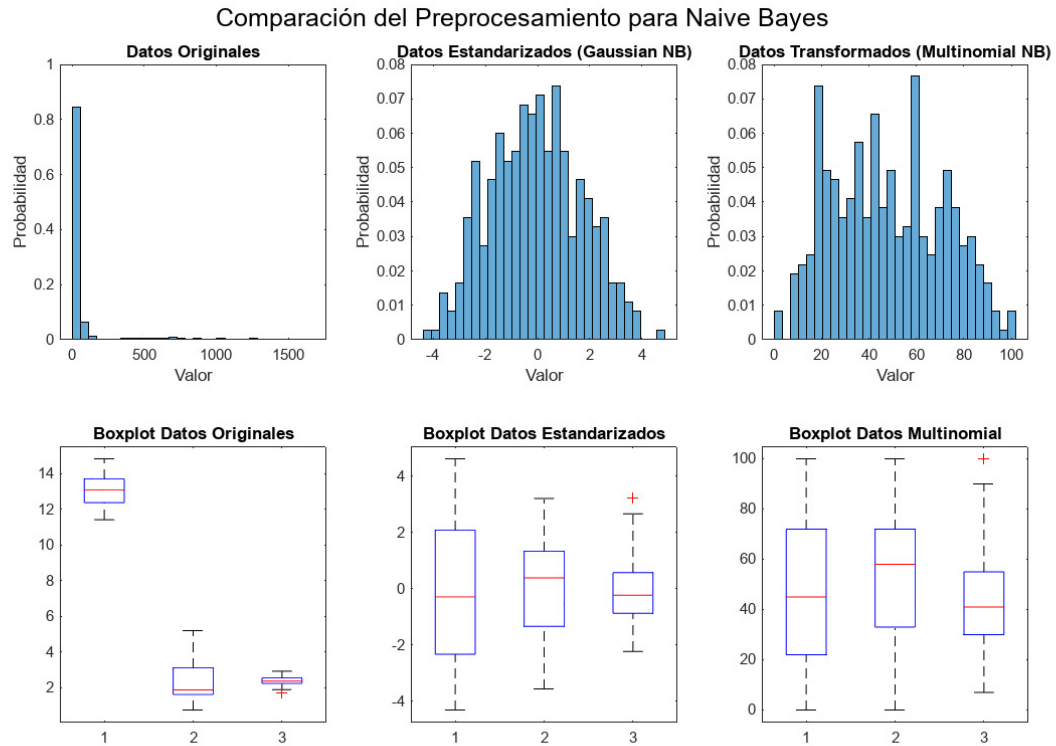
```

boxplot(X_train(:, 1:3));
title('Boxplot Datos Estandarizados');

subplot(2, 3, 6);
boxplot(X_train_multinom(:, 1:3));
title('Boxplot Datos Multinomial');

sgtitle('Comparación del Preprocesamiento para Naive Bayes');

```



```

%% =====
%% ENTRENAMIENTO DE MODELOS BAYESIANOS INGENUOS
%% =====
fprintf('\n=== ENTRENAMIENTO DE MODELOS NAIVE BAYES ===\n');

```

=== ENTRENAMIENTO DE MODELOS NAIVE BAYES ===

```

%% ENTRENAR MODELOS EN ESPACIO PCA
fprintf('\n=== ENTRENANDO MODELOS EN ESPACIO PCA ===\n');

```

=== ENTRENANDO MODELOS EN ESPACIO PCA ===


```
% Modelos Gaussianos en PCA
model_pca_classic = fitcnb(X_train(:,1:2), y_train, ...
    'DistributionNames', 'normal', 'Prior', 'uniform');

model_pca_empirical = fitcnb(X_train(:,1:2), y_train, ...
    'DistributionNames', 'normal', 'Prior', 'empirical');

model_pca_kernel = fitcnb(X_train(:,1:2), y_train, ...
    'DistributionNames', 'kernel', 'Prior', 'empirical');

% Para multinomial, usar los datos ya transformados
X_train_pca_multinom = X_train_multinom(:,1:2); % Primeras 2 componentes
X_test_pca_multinom = X_test_multinom(:,1:2);

model_pca_multinom_classic = fitcnb(X_train_pca_multinom, y_train, ...
    'DistributionNames', 'mn', 'Prior', 'uniform');

model_pca_multinom_empirical = fitcnb(X_train_pca_multinom, y_train, ...
    'DistributionNames', 'mn', 'Prior', 'empirical');
```

ANÁLISIS DE HIPERPARÁMETROS Y CONFIGURACIONES

GAUSSIAN NAIVE BAYES

- `DistributionNames = 'normal'`
 1. Asume que cada característica sigue distribución gaussiana
 2. Parámetros estimados: (media) y σ^2 (varianza) por clase y característica
 3. Apropiado cuando los datos son continuos y aproximadamente normales
- `DistributionNames = 'kernel'`
 1. Estimación no paramétrica usando densidad de kernel
 2. Más flexible, no asume forma específica de distribución
 3. `Kernel = 'normal'`: usa kernel gaussiano para suavizar
- `Prior`
 1. `'uniform'`: $P(C) = 1/K$ asume clases equiprobables
 2. `'empirical'`: $P(C) = n_c/n$ basado en frecuencias observadas

MULTINOMIAL NAIVE BAYES

- `DistributionNames = 'mn'`
 1. Asume distribución multinomial para cada característica
 2. Apropiado para datos de conteo/frecuencia

```
%% =====
%% 4. COMPARACIÓN DE MODELOS ENTRENADOS
%% =====
fprintf('\n=== INFORMACIÓN DE LOS MODELOS ===\n');
```

```
=== INFORMACIÓN DE LOS MODELOS ===
```

model_pca_classic

```
model_pca_classic =  
  ClassificationNaiveBayes  
    ResponseName: 'Y'  
    CategoricalPredictors: []  
    ClassNames: [1 2 3]  
    ScoreTransform: 'none'  
    NumObservations: 122  
    DistributionNames: {'normal' 'normal'}  
    DistributionParameters: {3x2 cell}
```

Properties, Methods

model_pca_kernel

```
model_pca_kernel =  
  ClassificationNaiveBayes  
    ResponseName: 'Y'  
    CategoricalPredictors: []  
    ClassNames: [1 2 3]  
    ScoreTransform: 'none'  
    NumObservations: 122  
    DistributionNames: {'kernel' 'kernel'}  
    DistributionParameters: {3x2 cell}  
    Kernel: {'normal' 'normal'}  
    Support: {'unbounded' 'unbounded'}  
    Width: [3x2 double]  
    Mu: []  
    Sigma: []
```

Properties, Methods

model_pca_empirical

```
model_pca_empirical =  
  ClassificationNaiveBayes  
    ResponseName: 'Y'  
    CategoricalPredictors: []  
    ClassNames: [1 2 3]  
    ScoreTransform: 'none'  
    NumObservations: 122  
    DistributionNames: {'normal' 'normal'}  
    DistributionParameters: {3x2 cell}
```

Properties, Methods

model_pca_multinom_classic

```
model_pca_multinom_classic =  
  ClassificationNaiveBayes  
      ResponseName: 'Y'  
      CategoricalPredictors: []  
      ClassNames: [1 2 3]  
      ScoreTransform: 'none'  
      NumObservations: 122  
      DistributionNames: 'mn'  
      DistributionParameters: {3x2 cell}
```

Properties, Methods

model_pca_multinom_empirical

```
model_pca_multinom_empirical =  
  ClassificationNaiveBayes  
      ResponseName: 'Y'  
      CategoricalPredictors: []  
      ClassNames: [1 2 3]  
      ScoreTransform: 'none'  
      NumObservations: 122  
      DistributionNames: 'mn'  
      DistributionParameters: {3x2 cell}
```

Properties, Methods

4. Visualización de resultados

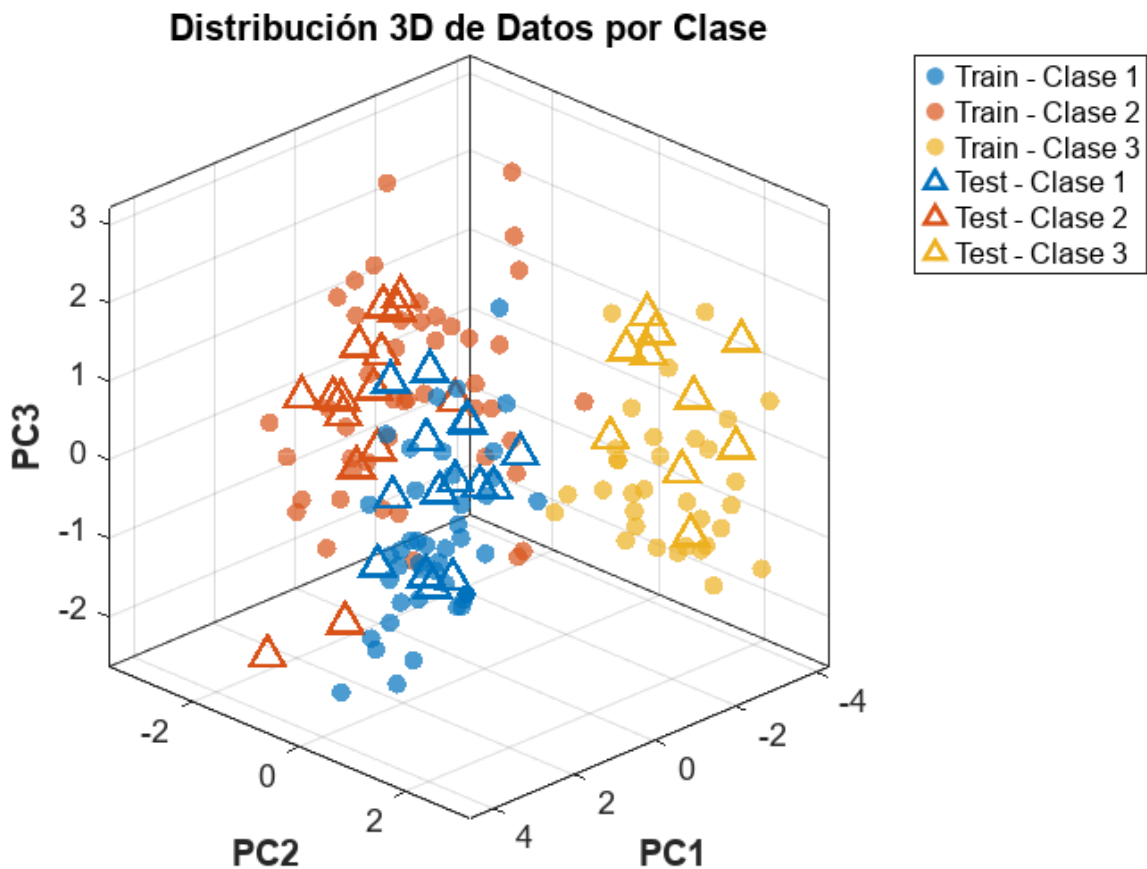
```
%% =====  
%% DISTRIBUCION DE CARACTERISTICAS  
%% =====  
figure  
hold on  
colors = lines(3); % Paleta de colores para 3 clases  
  
% Graficar puntos de entrenamiento  
for class = 1:3  
    idx = y_train == class;  
    scatter3(X_train(idx,1), X_train(idx,2), X_train(idx,3), ...  
            40, colors(class,:), 'filled', 'MarkerFaceAlpha', 0.7, ...  
            'DisplayName', ['Train - Clase ' num2str(class)]);  
end  
  
% Graficar puntos de prueba  
for class = 1:3  
    idx = y_test == class;
```

```

scatter3(X_test(idx,1), X_test(idx,2), X_test(idx,3), ...
        100, colors(class,:), '^', 'LineWidth', 1.5, ...
        'DisplayName', ['Test - Clase ' num2str(class)]);
end

% Etiquetas y mejoras
xlabel('PC1', 'FontSize', 12, 'FontWeight', 'bold')
ylabel('PC2', 'FontSize', 12, 'FontWeight', 'bold')
zlabel('PC3', 'FontSize', 12, 'FontWeight', 'bold')
title('Distribución 3D de Datos por Clase', 'FontSize', 14, 'FontWeight', 'bold')
grid on
box on
view(135, 25)
legend('Location', 'bestoutside')
set(gca, 'FontSize', 11)
axis tight

```



```

figure
hold on
colors = lines(3); % Paleta de colores para 3 clases

% Graficar puntos de entrenamiento

```

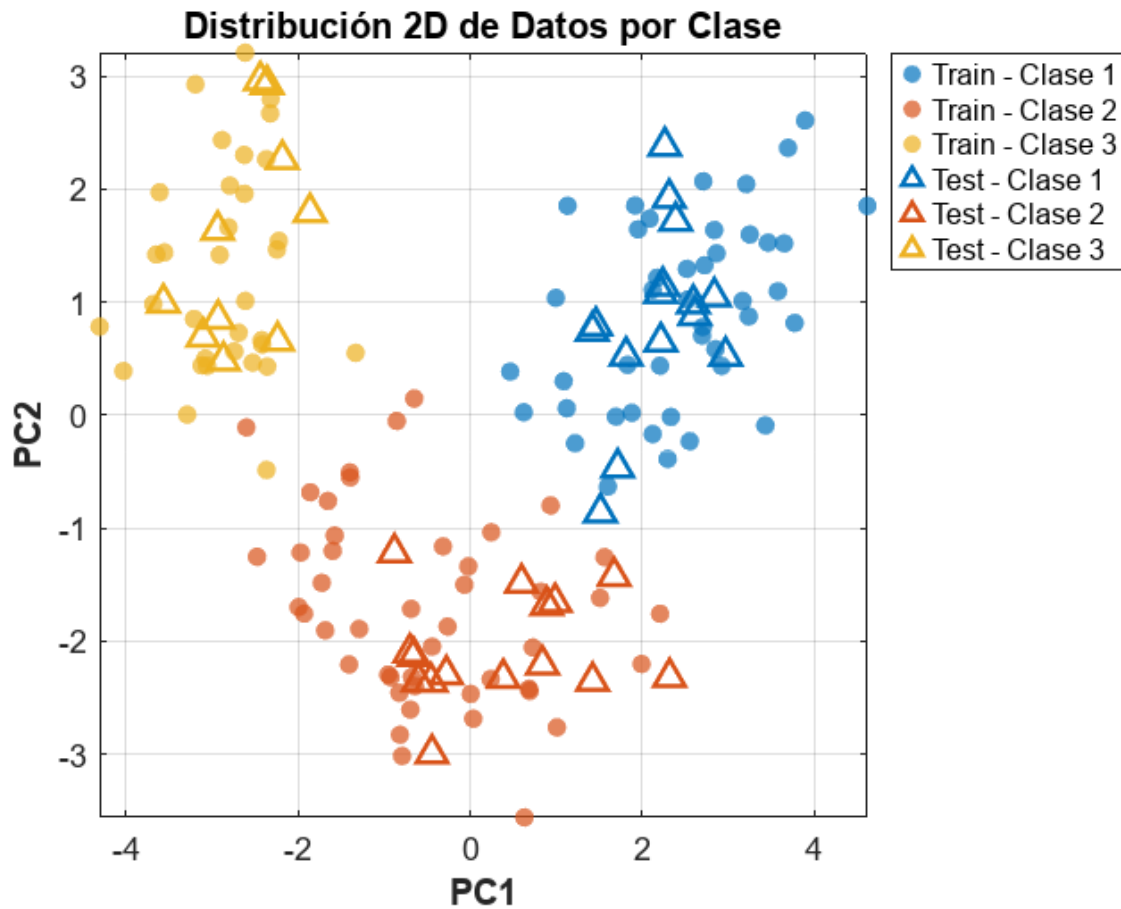
```

for class = 1:3
    idx = y_train == class;
    scatter(X_train(idx,1), X_train(idx,2), ...
        40, colors(class,:), 'filled', 'MarkerFaceAlpha', 0.7, ...
        'DisplayName', ['Train - Clase ' num2str(class)]);
end

% Graficar puntos de prueba
for class = 1:3
    idx = y_test == class;
    scatter(X_test(idx,1), X_test(idx,2), ...
        80, colors(class,:), '^', 'LineWidth', 1.5, ...
        'DisplayName', ['Test - Clase ' num2str(class)]);
end

% Etiquetas y mejoras
xlabel('PC1', 'FontSize', 12, 'FontWeight', 'bold')
ylabel('PC2', 'FontSize', 12, 'FontWeight', 'bold')
title('Distribución 2D de Datos por Clase', 'FontSize', 14, 'FontWeight', 'bold')
grid on
box on
legend('Location', 'bestoutside')
set(gca, 'FontSize', 11)
axis tight

```



```
%% =====
%% FRONTERAS DE DECISION
%% =====

models = { ...
    model_pca_classic, ...
    model_pca_empirical, ...
    model_pca_kernel, ...
    model_pca_multinom_classic, ...
    model_pca_multinom_empirical ...
};

modelName = { ...
    'NB Gaussiano - Prior Uniforme', ...
    'NB Gaussiano - Prior Empírico', ...
    'NB Kernel - Prior Empírico', ...
    'NB Multinomial - Prior Uniforme', ...
    'NB Multinomial - Prior Empírico' ...
};

% Preparar el rango global para PC1 y PC2
allX = [X_train(:,1:2); X_test(:,1:2)];
```

```

xMin = min(allX(:,1)) - 1;
xMax = max(allX(:,1)) + 1;
yMin = min(allX(:,2)) - 1;
yMax = max(allX(:,2)) + 1;

[xGrid, yGrid] = meshgrid(...
    linspace(xMin, xMax, 200), ...
    linspace(yMin, yMax, 200) ...
);
gridPoints = [xGrid(:), yGrid(:)];

% Colores para tres clases (1, 2 y 3)
classColors = lines(3);

% Crear figura con 1 fila y 3 columnas
figure('Units','normalized','Position',[0.1 0.1 0.85 0.5])

for i = 1:3
    mdl = models{i};
    ax = subplot(1, 3, i);
    hold(ax, 'on')

    % Predecir clases sobre la malla
    gridLabels = predict(mdl, gridPoints);
    gridLabels = reshape(gridLabels, size(xGrid));

    % Dibujar las regiones de decisión
    contourf(ax, xGrid, yGrid, gridLabels, 'LineColor', 'none')
    colormap(ax, parula(3))
    % Hacer el fondo semitransparente
    hContour = ax.Children(1);
    hContour.FaceAlpha = 0.3;

    % Graficar puntos de entrenamiento (círculos)
    for cls = 1:3
        idx_train = (y_train == cls);
        scatter( ...
            ax, ...
            X_train(idx_train,1), ...
            X_train(idx_train,2), ...
            36, classColors(cls,:), 'o', ...
            'filled', 'MarkerFaceAlpha', 0.8, ...
            'DisplayName', ['Train - Clase ' num2str(cls)] ...
        );
    end

    % Graficar puntos de prueba (triángulos con FaceColor igual a classColors)
    for cls = 1:3
        idx_test = (y_test == cls);
        scatter( ...
            ax, ...
            X_test(idx_test,1), ...
            X_test(idx_test,2), ...

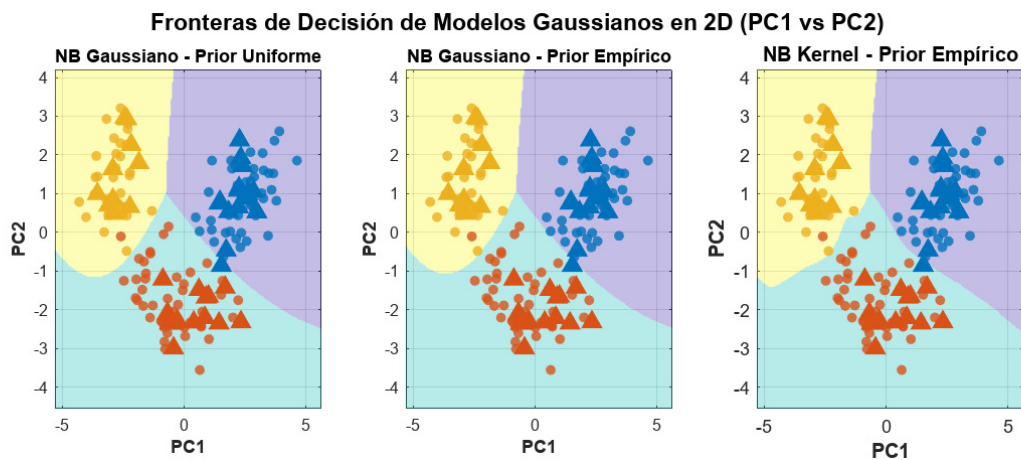
```

```

80, classColors(cls,:), '^', ...
'filled', 'MarkerFaceAlpha', 1, ...
'MarkerEdgeColor', classColors(cls,:), ...
'LineWidth', 1.5, ...
'DisplayName', ['Test - Clase ' num2str(cls)] ...
);
end

xlabel(ax, 'PC1', 'FontSize', 11, 'FontWeight', 'bold')
ylabel(ax, 'PC2', 'FontSize', 11, 'FontWeight', 'bold')
title(ax, modelNames{i}, 'FontSize', 12, 'FontWeight', 'bold')
axis(ax, [xMin xMax yMin yMax])
grid(ax, 'on')
box(ax, 'on')
set(ax, 'FontSize', 10)
hold(ax, 'off')
end
sgtitle('Fronteras de Decisión de Modelos Gaussianos en 2D (PC1 vs PC2)', ...
'FontSize', 14, 'FontWeight', 'bold')
grid on
box on
set(gca, 'FontSize', 11)
axis tight

```



```

%% =====
%% GRAFICO COMPARATIVO
%% =====

% Modelos gaussianos
models = {model_pca_classic, model_pca_empirical, model_pca_kernel};
titles = {'Gaussiano Clásico (Prior Uniforme)', 'Gaussiano Empírico', 'Kernel'};
train_data = {X_train(:,1:2), X_train(:,1:2), X_train(:,1:2)};
test_data = {X_test(:,1:2), X_test(:,1:2), X_test(:,1:2)};

accuracies_train = zeros(1, 3);
conf_matrices_train = cell(1, 3);

```



```

accuracies_test = zeros(1, 3);
conf_matrices_test = cell(1, 3);

for i = 1:3
    y_pred_train = predict(models{i}, train_data{i});
    accuracies_train(i) = mean(y_train == y_pred_train) * 100;
    conf_matrices_train{i} = confusionmat(y_train, y_pred_train);

    fprintf('%s: %.1f%% accuracy\n', titles{i}, accuracies_train(i));
end

```

Gaussiano Clásico (Prior Uniforme): 99.2% accuracy
 Gaussiano Empírico: 99.2% accuracy
 Kernel: 99.2% accuracy

```

for i = 1:3
    y_pred_test = predict(models{i}, test_data{i});
    accuracies_test(i) = mean(y_test == y_pred_test) * 100;
    conf_matrices_test{i} = confusionmat(y_test, y_pred_test);

    fprintf('%s: %.1f%% accuracy\n', titles{i}, accuracies_test(i));
end

```

Gaussiano Clásico (Prior Uniforme): 97.5% accuracy
 Gaussiano Empírico: 97.5% accuracy
 Kernel: 97.5% accuracy

```

% Gráfico comparativo
figure('Position', [200, 200, 800, 400]);

subplot(1, 2, 1);
bar(accuracies_train, 'FaceColor', [0.8 0.4 0.2]);
title('Comparación de Accuracy (train)', 'FontSize', 14, 'FontWeight', 'bold');
ylabel('Accuracy (%)', 'FontSize', 12);
xlabel('Modelos Gaussianos', 'FontSize', 12);
set(gca, 'XTickLabel', {'Clásico', 'Empírico', 'Kernel'});
ylim([0, 100]);
grid on;

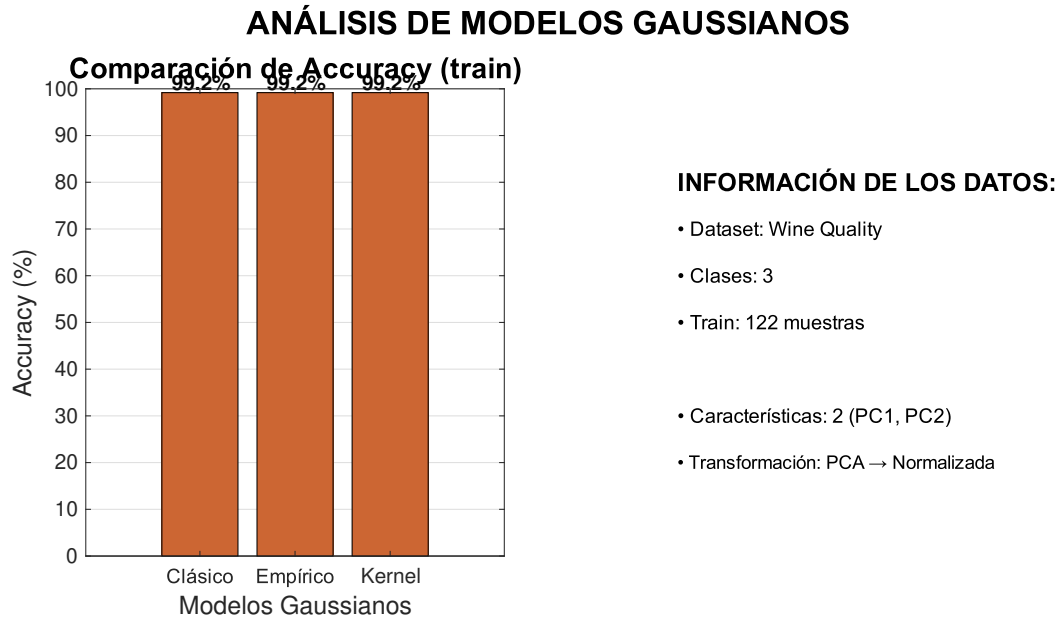
% Añadir valores encima de las barras
for i = 1:3
    text(i, accuracies_train(i) + 2, sprintf('%.1f%%', accuracies_train(i)), ...
        'HorizontalAlignment', 'center', 'FontWeight', 'bold');
end

% Mostrar información adicional
subplot(1, 2, 2);
text(0.1, 0.8, 'INFORMACIÓN DE LOS DATOS:', 'FontSize', 12, 'FontWeight', 'bold');
text(0.1, 0.7, sprintf('• Dataset: Wine Quality'), 'FontSize', 10);
text(0.1, 0.6, sprintf('• Clases: %d', length(unique(y_train))), 'FontSize', 10);
text(0.1, 0.5, sprintf('• Train: %d muestras', length(y_train)), 'FontSize', 10);
text(0.1, 0.3, sprintf('• Características: 2 (PC1, PC2)'), 'FontSize', 10);

```

```
text(0.1, 0.2, sprintf('• Transformación: PCA → Normalizada'), 'FontSize', 9);
axis off;

sgtitle('ANÁLISIS DE MODELOS GAUSSIANOS', 'FontSize', 16, 'FontWeight', 'bold');
```



```
figure('Position', [200, 200, 800, 400]);

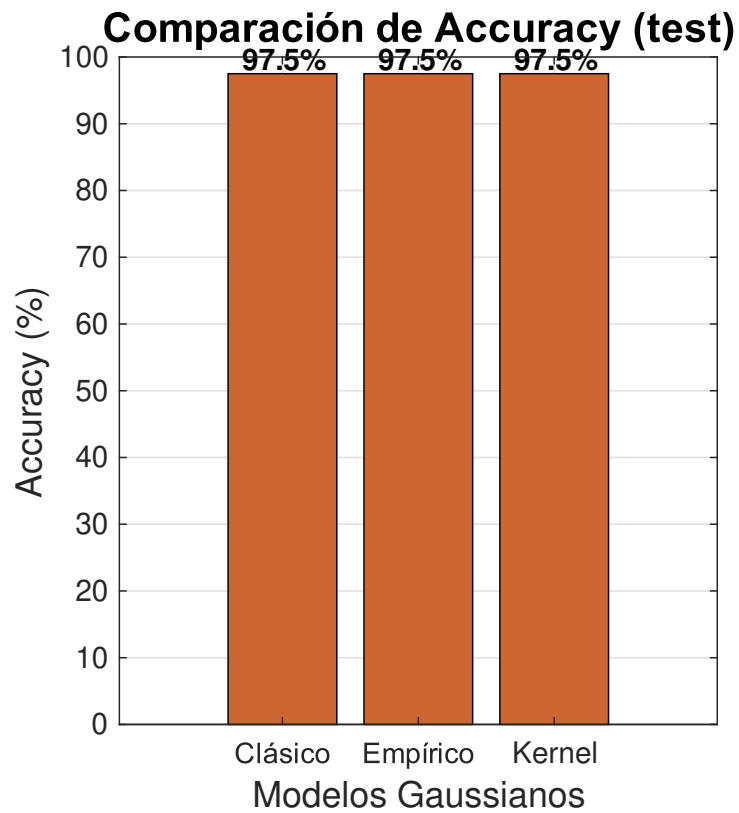
subplot(1, 2, 1);
bar(accuracies_test, 'FaceColor', [0.8 0.4 0.2]);
title('Comparación de Accuracy (test)', 'FontSize', 14, 'FontWeight', 'bold');
ylabel('Accuracy (%)', 'FontSize', 12);
xlabel('Modelos Gaussianos', 'FontSize', 12);
set(gca, 'XTickLabel', {'Clásico', 'Empírico', 'Kernel'});
ylim([0, 100]);
grid on;

% Añadir valores encima de las barras
for i = 1:3
    text(i, accuracies_test(i) + 2, sprintf('%.1f%%', accuracies_test(i)), ...
        'HorizontalAlignment', 'center', 'FontWeight', 'bold');
end

% Mostrar información adicional
subplot(1, 2, 2);
text(0.1, 0.8, 'INFORMACIÓN DE LOS DATOS:', 'FontSize', 12, 'FontWeight', 'bold');
text(0.1, 0.7, sprintf('• Dataset: Wine Quality'), 'FontSize', 10);
text(0.1, 0.6, sprintf('• Clases: %d', length(unique(y_train))), 'FontSize', 10);
text(0.1, 0.4, sprintf('• Test: %d muestras', length(y_test)), 'FontSize', 10);
text(0.1, 0.3, sprintf('• Características: 2 (PC1, PC2)'), 'FontSize', 10);
text(0.1, 0.2, sprintf('• Transformación: PCA → Normalizada'), 'FontSize', 9);
axis off;
```

```
sgtitle('ANÁLISIS DE MODELOS GAUSSIANOS', 'FontSize', 16, 'FontWeight', 'bold');
```

ANÁLISIS DE MODELOS GAUSSIANOS



INFORMACIÓN

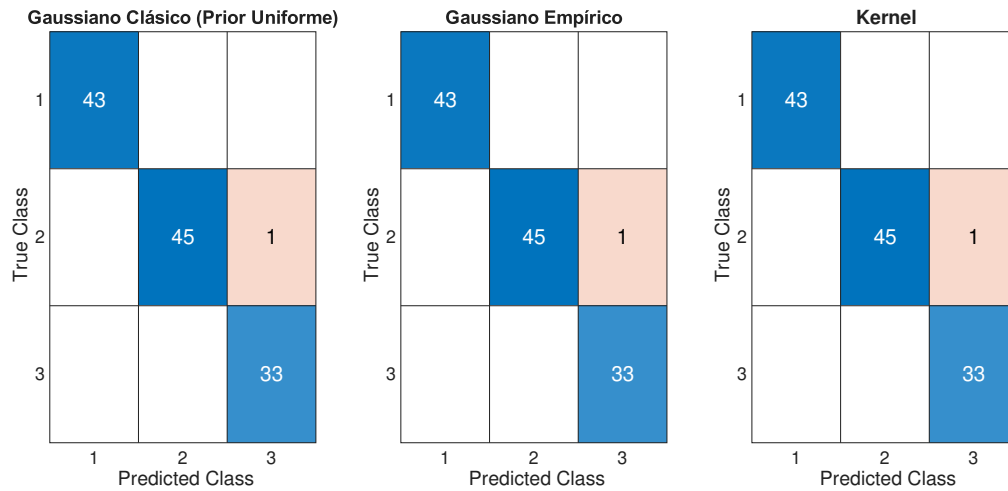
- Dataset: Wir
- Clases: 3
- Test: 40 muestras
- Características: 10
- Transformación: Normalización

```
%% MATRICES DE CONFUSIÓN
figure('Position', [300, 300, 800, 350]);

for i = 1:3
    subplot(1, 3, i);
    y_pred_train = predict(models{i}, train_data{i});
    confusionchart(y_train, y_pred_train, 'Title', titles{i}, 'FontSize', 10);
end

sgtitle('MATRICES DE CONFUSIÓN - MODELOS GAUSSIANOS (train)', 'FontSize', 14, 'FontWeight', 'bold');
```

MATRICES DE CONFUSIÓN - MODELOS GAUSSIANOS (train)

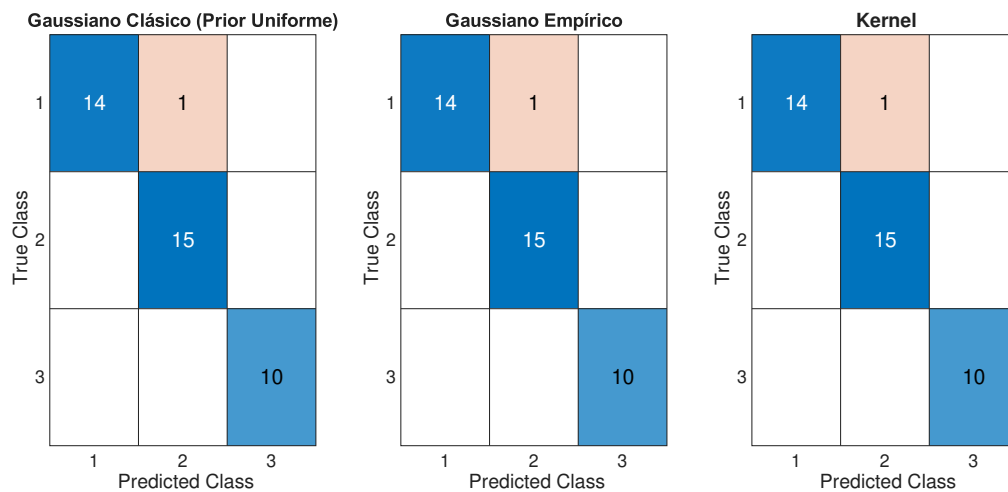


```
%% MATRICES DE CONFUSIÓN
figure('Position', [300, 300, 800, 350]);

for i = 1:3
    subplot(1, 3, i);
    y_pred_test = predict(models{i}, test_data{i});
    confusionchart(y_test, y_pred_test, 'Title', titles{i}, 'FontSize', 10);
end

sgtitle('MATRICES DE CONFUSIÓN - MODELOS GAUSSIANOS (test)', 'FontSize', 14, 'FontWeight', 'bold');
```

MATRICES DE CONFUSIÓN - MODELOS GAUSSIANOS (test)



```
% Modelos multinomiales
models = {model_pca_multinom_classic, model_pca_multinom_empirical};
titles = {'Multinomial Clásico (Prior Uniforme)', 'Multinomial Empírico'};
train_data = {X_train_pca_multinom, X_train_pca_multinom};
```

```

test_data = {X_test_pca_multinom, X_test_pca_multinom};

accuracies_multinom_train = zeros(1, 2);
conf_matrices_train = cell(1, 2);

accuracies_multinom_test = zeros(1, 2);
conf_matrices_test = cell(1, 2);

for i = 1:2
    y_pred_train = predict(models{i}, train_data{i});
    accuracies_multinom_train(i) = mean(y_train == y_pred_train) * 100;
    conf_matrices_train{i} = confusionmat(y_train, y_pred_train);

    fprintf('%s: %.1f%% accuracy\n', titles{i}, accuracies_multinom_train(i));
end

```

Multinomial Clásico (Prior Uniforme): 79.5% accuracy
 Multinomial Empírico: 79.5% accuracy

```

for i = 1:2
    y_pred_test = predict(models{i}, test_data{i});
    accuracies_multinom_test(i) = mean(y_test == y_pred_test) * 100;
    conf_matrices_test{i} = confusionmat(y_test, y_pred_test);

    fprintf('%s: %.1f%% accuracy\n', titles{i}, accuracies_multinom_test(i));
end

```

Multinomial Clásico (Prior Uniforme): 90.0% accuracy
 Multinomial Empírico: 90.0% accuracy

```

% Gráfico comparativo
figure('Position', [200, 200, 800, 400]);

subplot(1, 2, 1);
bar(accuracies_multinom_train, 'FaceColor', [0.8 0.4 0.2]);
title('Comparación de Accuracy (train)', 'FontSize', 14, 'FontWeight', 'bold');
ylabel('Accuracy (%)', 'FontSize', 12);
xlabel('Modelos Multinomiales', 'FontSize', 12);
set(gca, 'XTickLabel', {'Clásico', 'Empírico'});
ylim([0, 100]);
grid on;

% Añadir valores encima de las barras
for i = 1:2
    text(i, accuracies_multinom_train(i) + 2, sprintf('%.1f%%', accuracies_multinom_train(i)), ...
        'HorizontalAlignment', 'center', 'FontWeight', 'bold');
end

% Mostrar información adicional
subplot(1, 2, 2);
text(0.1, 0.8, 'INFORMACIÓN DE LOS DATOS:', 'FontSize', 12, 'FontWeight', 'bold');
text(0.1, 0.7, sprintf('• Dataset: Wine Quality'), 'FontSize', 10);

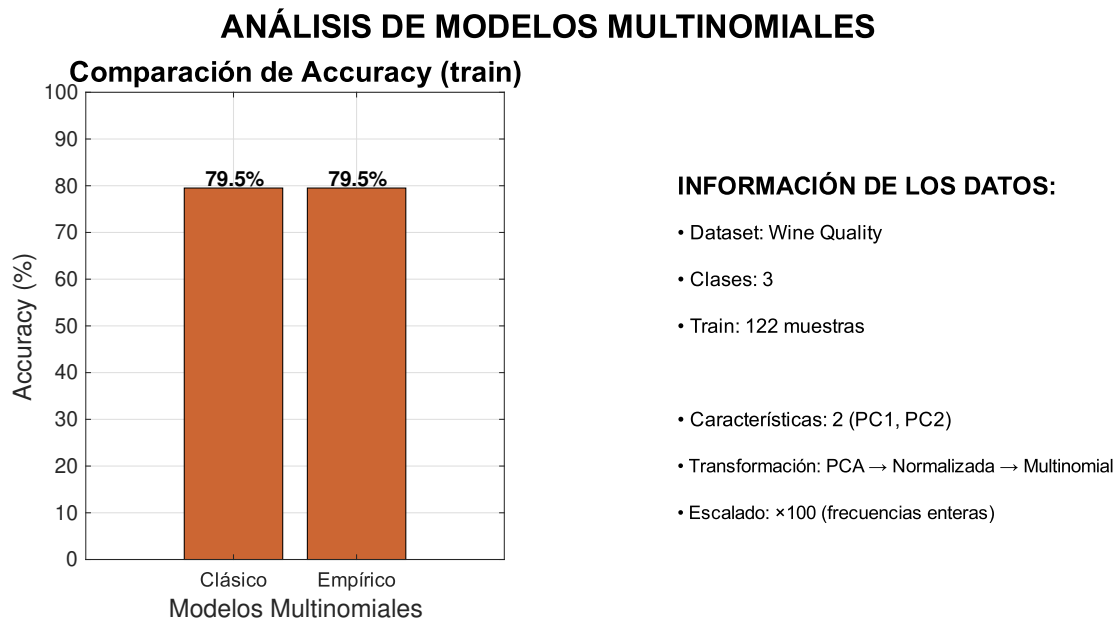
```

```

text(0.1, 0.6, sprintf('• Clases: %d', length(unique(y_train))), 'FontSize', 10);
text(0.1, 0.5, sprintf('• Train: %d muestras', length(y_train)), 'FontSize', 10);
text(0.1, 0.3, sprintf('• Características: 2 (PC1, PC2)'), 'FontSize', 10);
text(0.1, 0.2, sprintf('• Transformación: PCA → Normalizada → Multinomial'), 'FontSize', 9);
text(0.1, 0.1, sprintf('• Escalado: ×%d (frecuencias enteras)', scaling_factor), 'FontSize', 9);
axis off;

sgtitle('ANÁLISIS DE MODELOS MULTINOMIALES', 'FontSize', 16, 'FontWeight', 'bold');

```



```

%%
figure('Position', [200, 200, 800, 400]);

subplot(1, 2, 1);
bar(accuracies_multinom_test, 'FaceColor', [0.8 0.4 0.2]);
title('Comparación de Accuracy (test)', 'FontSize', 14, 'FontWeight', 'bold');
ylabel('Accuracy (%)', 'FontSize', 12);
xlabel('Modelos Multinomiales', 'FontSize', 12);
set(gca, 'XTickLabel', {'Clásico', 'Empírico'});
ylim([0, 100]);
grid on;

% Añadir valores encima de las barras
for i = 1:2
    text(i, accuracies_multinom_test(i) + 2, sprintf('%.1f%%', accuracies_multinom_test(i)), ...
        'HorizontalAlignment', 'center', 'FontWeight', 'bold');
end

% Mostrar información adicional
subplot(1, 2, 2);
text(0.1, 0.8, 'INFORMACIÓN DE LOS DATOS:', 'FontSize', 12, 'FontWeight', 'bold');
text(0.1, 0.7, sprintf('• Dataset: Wine Quality'), 'FontSize', 10);

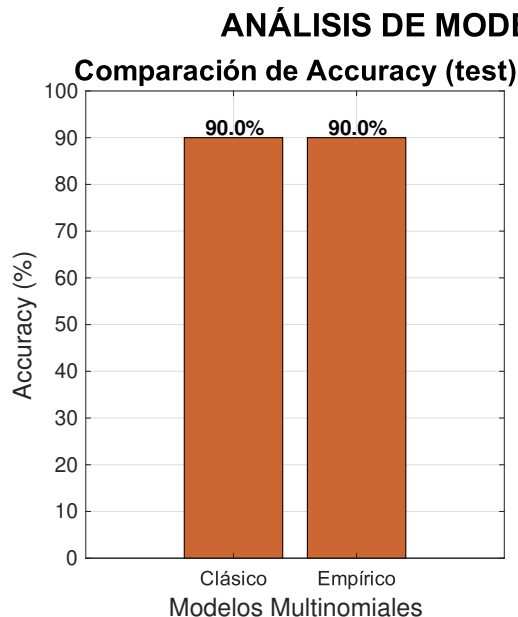
```

```

text(0.1, 0.6, sprintf('• Clases: %d', length(unique(y_train))), 'FontSize', 10);
text(0.1, 0.4, sprintf('• Test: %d muestras', length(y_test)), 'FontSize', 10);
text(0.1, 0.3, sprintf('• Características: 2 (PC1, PC2)'), 'FontSize', 10);
text(0.1, 0.2, sprintf('• Transformación: PCA → Normalizada → Multinomial'), 'FontSize', 9);
text(0.1, 0.1, sprintf('• Escalado: ×%d (frecuencias enteras)', scaling_factor), 'FontSize', 9);
axis off;

sgtitle('ANÁLISIS DE MODELOS MULTINOMIALES', 'FontSize', 16, 'FontWeight', 'bold');

```



INFORMACIÓN DE LOS DATOS:

- Dataset: Wine Quality
- Clases: 3
- Test: 40 muestras
- Características: 2 (PC1, PC2)
- Transformación: PCA → Normalizada → Multinomial
- Escalado: ×100 (frecuencias enteras)

```

%% MATRICES DE CONFUSIÓN
figure('Position', [300, 300, 800, 350]);

for i = 1:2
    subplot(1, 2, i);
    y_pred_train = predict(models{i}, train_data{i});
    confusionchart(y_train, y_pred_train, 'Title', titles{i}, 'FontSize', 10);
end

sgtitle('MATRICES DE CONFUSIÓN - MODELOS MULTINOMIALES (train)', 'FontSize', 14, 'FontWeight', 'bold')

```

MATRICES DE CONFUSIÓN - MODELOS MULTINOMIALES (train)

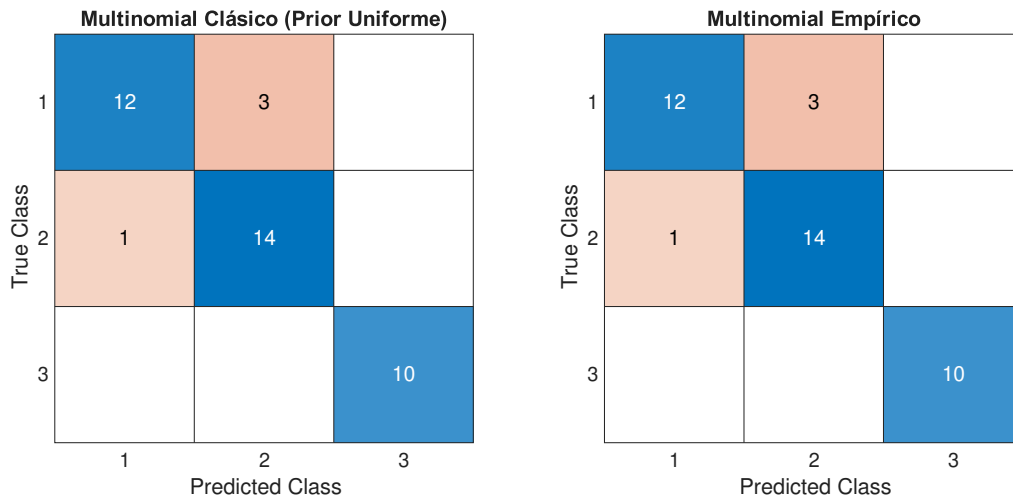


```
figure('Position', [300, 300, 800, 350]);
```

```
for i = 1:2
    subplot(1, 2, i);
    y_pred_test = predict(models{i}, test_data{i});
    confusionchart(y_test, y_pred_test, 'Title', titles{i}, 'FontSize', 10);
end
```

```
sgtitle('MATRICES DE CONFUSIÓN - MODELOS MULTINOMIALES (test)', 'FontSize', 14, 'FontWeight', 'bold')
```

MATRICES DE CONFUSIÓN - MODELOS MULTINOMIALES (test)



```
%% RESUMEN
```

```
[best_acc, best_idx] = max(accuracies_test);
fprintf('Mejor modelo gaussiano: %s\n', titles{best_idx});
```

```
Mejor modelo gaussiano: Multinomial Clásico (Prior Uniform)
```



```
fprintf('Mejor accuracy: %.1f%%\n', best_acc);
```

Mejor accuracy: 97.5%

```
[best_acc, best_idx] = max(accuracies_multinom_test);  
fprintf('Mejor modelo multinomial: %s\n', titles{best_idx});
```

Mejor modelo multinomial: Multinomial Clásico (Prior Uniforme)

```
fprintf('Mejor accuracy: %.1f%%\n', best_acc);
```

Mejor accuracy: 90.0%

5. Análisis de resultados

En el análisis de modelos de clasificación aplicados al conjunto de datos clásico de vinos, se compararon distintas variantes del clasificador Naive Bayes: Gaussiano (con prior uniforme y empírico), Gaussiano con kernel, y Multinomial (también con prior uniforme y empírico). Los resultados obtenidos muestran un rendimiento notablemente alto para los modelos gaussianos, con una exactitud del 99.2% en el conjunto de entrenamiento y del 97.5% en el conjunto de prueba, sin diferencias entre las versiones clásica, empírica o con kernel, lo que sugiere una gran estabilidad y adecuación del supuesto de normalidad a este conjunto de datos. Por otro lado, el modelo Naive Bayes Multinomial mostró un comportamiento menos consistente: aunque obtuvo una exactitud más baja en el entrenamiento (79.5%), logró un rendimiento aceptable en el conjunto de prueba con un 90.0% de exactitud. Esta discrepancia podría deberse a que el modelo multinomial, más adecuado para datos discretos como texto o conteos, no se ajusta tan bien a las características continuas del conjunto de vinos. En conjunto, los resultados indican que el modelo Naive Bayes Gaussiano es el más apropiado para este caso, mostrando tanto alta precisión como buena generalización.

Ejercicio 3: Árboles de decisión sobre el conjunto de subespecies de flor iris

Practica4_pt3

May 30, 2025

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.decomposition import PCA
```

CARGA Y LIMPIEZA DE DATOS

```
[2]: iris = load_iris()
X, y = iris.data, iris.target

# Convertir a DataFrame para verificar faltantes y duplicados
df = pd.DataFrame(X, columns=iris.feature_names)
df['target'] = y

# i. Verificar valores faltantes y eliminar duplicados
print("Valores faltantes por columna:\n", df.isnull().sum())
df = df.drop_duplicates()
print(f"\nNúmero de muestras tras eliminar duplicados: {df.shape[0]} (original: {load_iris().data.shape[0]})")

# Separar X e y tras limpieza
X_clean = df[iris.feature_names].values
y_clean = df['target'].values
```

```
Valores faltantes por columna:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target              0
```

dtype: int64

Número de muestras tras eliminar duplicados: 149 (original: 150)

PREPROCESAMIENTO DE DATOS

```
[3]: # i. Estandarizar características
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_clean)

# ii. Dividir en 80% entrenamiento y 20% prueba
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_clean,
    ↪test_size=0.2, random_state=42, stratify=y_clean)
```

ENTRENAMIENTO DEL MODELO DE ARBOLES DE DECISION

```
[4]: # i. Profundidad máxima = 3
clf = DecisionTreeClassifier(max_depth=3, random_state=42)
clf.fit(X_train, y_train)

# ii. Evaluación
y_pred = clf.predict(X_test)
accuracy_test = accuracy_score(y_test, y_pred)
accuracy_train = accuracy_score(y_train, clf.predict(X_train))

print(f"\nExactitud (Accuracy) en conjunto de prueba: {accuracy_test:.2f}")
print(f"Exactitud (Accuracy) en conjunto de entrenamiento: {accuracy_train:.
    ↪2f}")

cm = confusion_matrix(y_test, y_pred)
```

Exactitud (Accuracy) en conjunto de prueba: 0.97

Exactitud (Accuracy) en conjunto de entrenamiento: 0.98

VISUALIZACION DE RESULTADOS

```
[5]: # i. Estructura del árbol con reglas de decisión
plt.figure(figsize=(10, 6))
```

```

plot_tree(clf, feature_names=iris.feature_names, class_names=iris.target_names,
    ↪filled=True, rounded=True)
plt.title("Árbol de Decisión (max_depth=3)")
plt.show()

# ii. Fronteras de decisión (utilizamos PCA para reducir a 2 dimensiones para
    ↪graficar)
pca_2d = PCA(n_components=2, random_state=42)
X_pca2d = pca_2d.fit_transform(X_scaled)

# Dividir los datos PCA en entreno/prueba para visualización
Xp_train, Xp_test, yp_train, yp_test = train_test_split(X_pca2d, y_clean,
    ↪test_size=0.2, random_state=42, stratify=y_clean)

# Entrenar un árbol sobre las componentes PCA (solo para visualización de
    ↪fronteras)
clf_pca = DecisionTreeClassifier(max_depth=3, random_state=42)
clf_pca.fit(Xp_train, yp_train)

# Definir malla para el plano PCA
x_min, x_max = X_pca2d[:, 0].min() - 1, X_pca2d[:, 0].max() + 1
y_min, y_max = X_pca2d[:, 1].min() - 1, X_pca2d[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300))
grid_points = np.c_[xx.ravel(), yy.ravel()]

# Predecir en la malla
Z = clf_pca.predict(grid_points).reshape(xx.shape)

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')

# Puntos de entrenamiento
plt.scatter(Xp_train[:, 0], Xp_train[:, 1], c=yp_train, cmap='coolwarm',
    ↪edgecolors='k', label='Entrenamiento', marker='o')
# Puntos de prueba
plt.scatter(Xp_test[:, 0], Xp_test[:, 1], c=yp_test, cmap='coolwarm',
    ↪edgecolors='k', label='Prueba', marker='s')

plt.xlabel('PCA 1')
plt.ylabel('PCA 2')
plt.title('Fronteras de decisión (árbol en PCA-2D)')
plt.legend()
plt.show()

# iii. Distribución espacial en 3 dimensiones (usar PCA para 3 componentes)

```

```

pca_3d = PCA(n_components=3, random_state=42)
X_pca3d = pca_3d.fit_transform(X_scaled)

fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(
    X_pca3d[:, 0], X_pca3d[:, 1], X_pca3d[:, 2],
    c=y_clean, cmap='viridis', edgecolors='k', s=50
)
ax.set_xlabel('PCA 1')
ax.set_ylabel('PCA 2')
ax.set_zlabel('PCA 3')
ax.set_title('Distribución del conjunto Iris en 3D (PCA)')

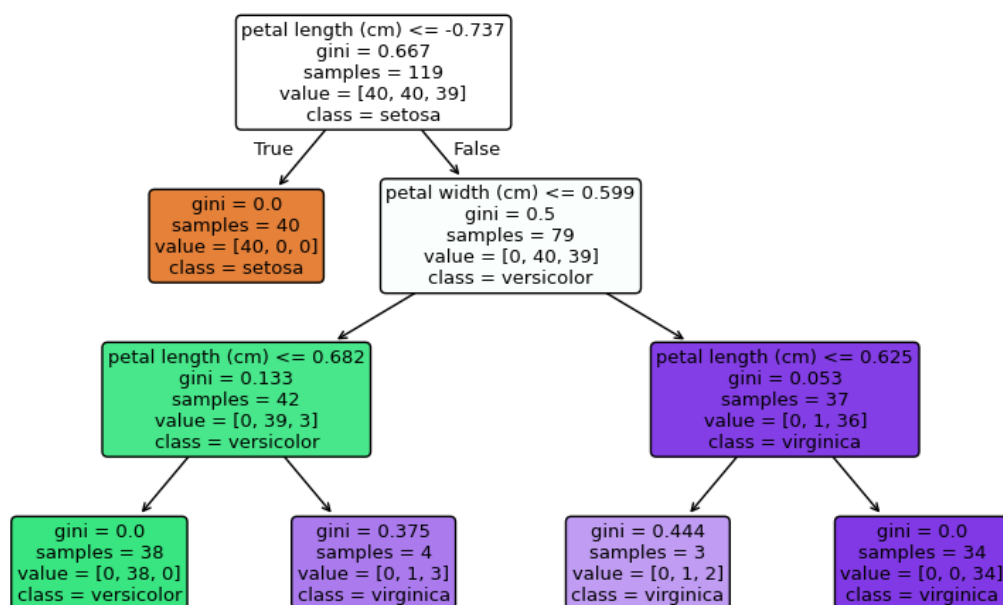
# Crear leyenda de clases
legend_handles = []
for idx, target_name in enumerate(iris.target_names):
    legend_handles.append(
        plt.Line2D([0], [0], marker='o', color='w', label=target_name,
                    markerfacecolor=plt.cm.viridis(idx / len(iris.
    ↪target_names))), markersize=8, markeredgecolor='k')
    )
ax.legend(handles=legend_handles, title='Clase')
plt.show()

# Mostrar la matriz de confusión en formato DataFrame
cm_df = pd.DataFrame(cm,
                      index=[f"True {name}" for name in iris.target_names],
                      columns=[f"Pred {name}" for name in iris.target_names])

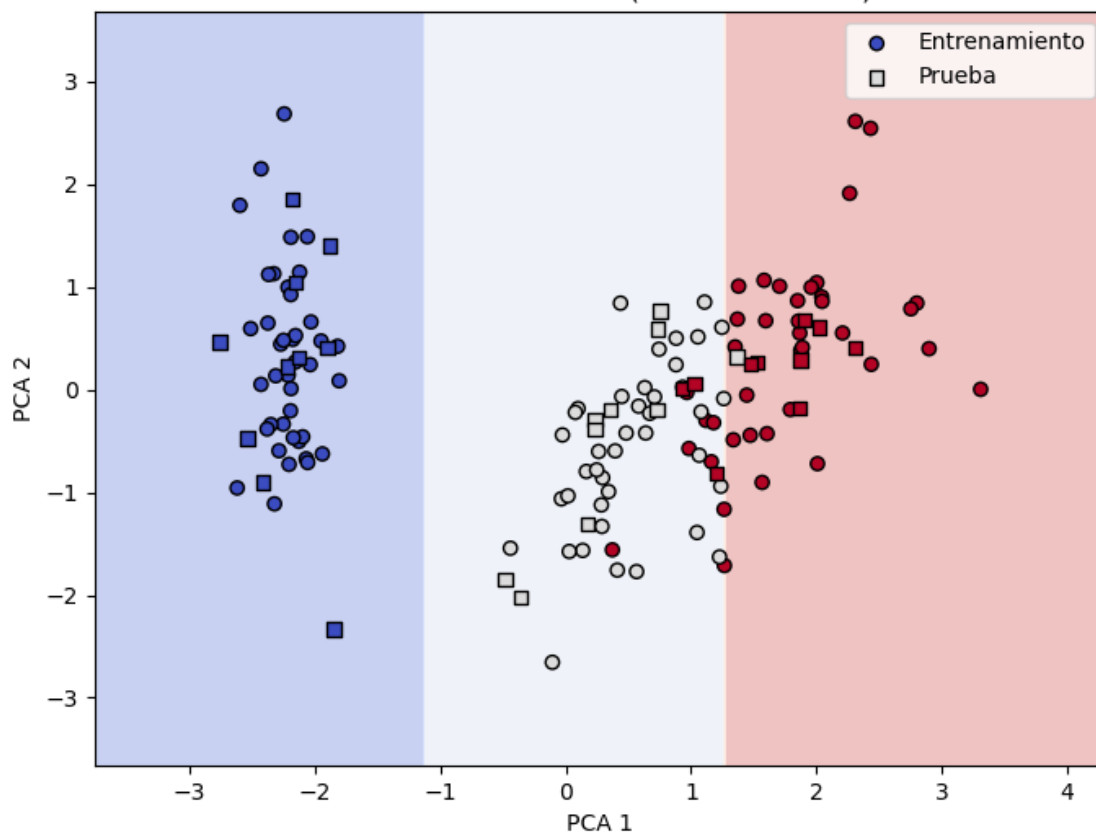
# Visualizar matriz de confusión como heatmap
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names,
            yticklabels=iris.target_names)
plt.title("Matriz de Confusión - Árbol de Decisión")
plt.xlabel("Predicho")
plt.ylabel("Real")
plt.show()

```

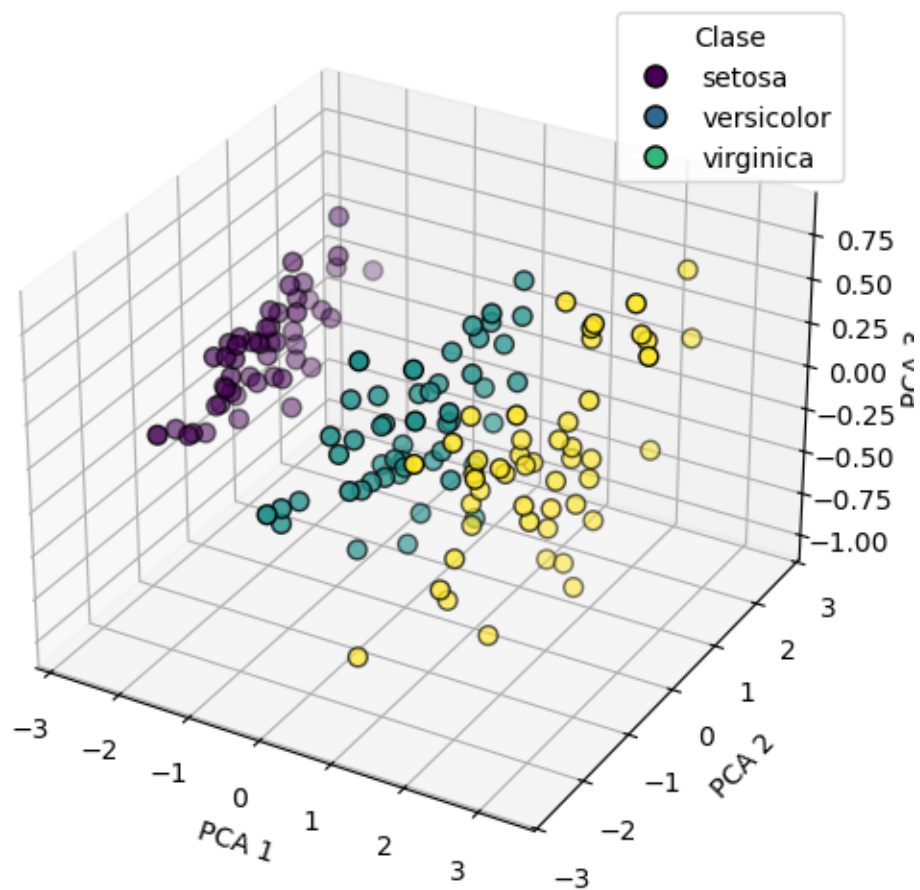
Árbol de Decisión (max_depth=3)

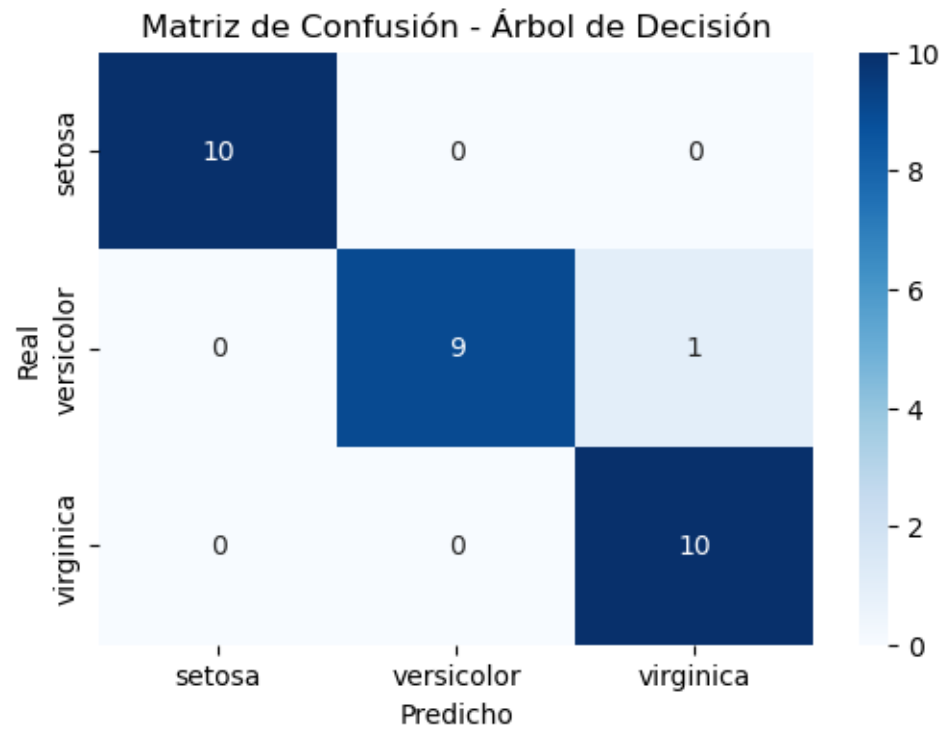


Fronteras de decisión (árbol en PCA-2D)



Distribución del conjunto Iris en 3D (PCA)





Conclusiones

Con base en el análisis y resultados presentados en esta práctica, se pueden extraer varias conclusiones relevantes sobre el desempeño y aplicabilidad de distintos métodos supervisados.

En primer lugar, se observó que tanto KNN como DMIN ofrecen resultados razonables en la clasificación del conjunto de datos de cáncer de mama, aunque DMIN mostró una mejor exactitud después de la reducción de dimensionalidad mediante PCA. Esto sugiere que modelos basados en centroides pueden beneficiarse más de la reducción de ruido y redundancia en los datos. Además, la visualización en 2D y 3D permitió apreciar de forma clara cómo se agrupan las clases y cómo responden los modelos ante distintas representaciones del espacio de características.

Por otro lado, los clasificadores basados en Naive Bayes evidenciaron que el supuesto de distribución de las características impacta significativamente el rendimiento. El modelo Gaussiano, tanto en sus versiones clásica como empírica y con kernel, alcanzó altos niveles de precisión, demostrando ser altamente adecuado para el conjunto de datos Wine, caracterizado por variables continuas y aproximadamente normales. En contraste, el modelo Multinomial, aunque útil en datos discretos, mostró menor exactitud, lo cual refuerza la importancia de seleccionar el modelo adecuado según la naturaleza del problema y las características de los datos.

En cuanto a los clasificadores SVM, se confirmó su eficacia para encontrar fronteras de decisión óptimas en espacios transformados, particularmente cuando se aplicó PCA para facilitar la separación lineal. La variación del hiperparámetro C permitió observar un pequeño impacto en la precisión, lo que indica cierta robustez del modelo frente a cambios moderados en su configuración.

Finalmente, el análisis con árboles de decisión sobre el conjunto Iris destacó la facilidad interpretativa de este modelo, al permitir representar las reglas de clasificación de manera explícita. Aunque su exactitud no fue tan elevada como la de otros métodos en algunos casos, su capacidad para describir relaciones jerárquicas entre características lo convierte en una herramienta valiosa, especialmente cuando se requiere transparencia en la toma de decisiones.

En conjunto, la práctica permitió contrastar diversos modelos, visualizaciones y técnicas de preprocesamiento, subrayando la importancia del análisis exploratorio, la reducción de dimensionalidad y la selección adecuada de clasificadores. Este tipo de estudios comparativos es esencial para desarrollar una intuición sólida sobre cuándo y cómo aplicar cada técnica en problemas reales de ciencia de datos.

Referencias

- Tolstikhin, Ilya O., et al. *Mlp-mixer: An all-mlp architecture for vision*. Advances in neural information processing systems 34 (2021): 24261-24272.
- Wu, Baoyuan, et al. *Defenses in adversarial machine learning: A survey*. arXiv preprint arXiv:2312.08890 (2023).
- Liu, Xiao, et al. *Self-supervised learning: Generative or contrastive*. IEEE transactions on knowledge and data engineering 35.1 (2021): 857-876.
- Goodfellow, Ian. *Deep learning*. (2016).
- Murphy, Kevin P. *Machine learning: a probabilistic perspective*. MIT press, 2012.
- Wolberg, W., Mangasarian, O., Street, N., Street, W. (1993). *Breast Cancer Wisconsin (Diagnostic)* [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5DW2B>
- Cortez, P., Cerdeira, A., Almeida, F., Matos, T., Reis, J. (2009). *Wine Quality* [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C56S3T>
- Fisher, R. (1936). *Iris* [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C56C76>